Diseño de infraestructura de red de agua

Trabajo Práctico Integrador — Lógica y Teoría de Grafos

Integrantes: Matías Rubio · Ciro Pregot

Fecha de entrega: 08/11/2026

Repositorio: https://github.com/Matuti2611/Diseno-de-Infraestructura-de-Red

API: https://infraestructura-red-api.onrender.com/docs

Frontend en Vercel: https://diseno-de-infraestructura-de-red.vercel.app

Resumen

Una cooperativa de agua debe decidir qué tendidos construir para conectar 16 barrios con una planta al menor costo y verificar si esa red alcanza para la demanda dentro de cinco años. El trabajo encadena un modelo de regresión que proyecta el consumo de cada barrio, el algoritmo de Kruskal para obtener la red de menor costo (73.066,05 u. c.) y el algoritmo de Edmonds–Karp para medir cuánta demanda puede abastecer esa red. Una aplicación web (FastAPI y React) permite simular escenarios de crecimiento (Base, +10 % y +30 %) y dos casos de capacidades (Original y Progresivo).

Con las capacidades del caso Original, la red de menor costo no cubre la demanda en ninguno de los tres escenarios: el tendido Z01 — Z05 (1.200 m³/día) queda saturado y el déficit va de 731,74 a 792,68 m³/día. Con el caso Progresivo en +10 %, la red cubre la demanda y el único tramo crítico pasa a ser la entrada de la planta, al 99,9 % de su capacidad. La aplicación informa además el suministro y el déficit de cada barrio.

1. Descripción del problema y decisiones de diseño

Una cooperativa de servicios de agua debe planificar el tendido de su red en una zona nueva. Necesita decidir qué conexiones construir para llegar a todos los barrios con el menor costo posible y, además, verificar si la capacidad de esa red alcanza para la demanda que habrá dentro de cinco años.

El problema tiene dos partes que se resuelven con herramientas distintas, pero encadenadas:

1. **Proyectar la demanda futura de cada barrio.** No es un dato conocido: hay que estimarlo a partir de características actuales (población, consumo, tipo de zona, crecimiento esperado). Se resuelve con un modelo de regresión.
2. **Diseñar y validar la red.** Con el grafo de tendidos posibles se busca la red de menor costo que conecte todo (árbol recubridor mínimo) y luego se comprueba, con flujo máximo, cuánta de la demanda proyectada puede transportar esa red.

Por qué un grafo. Los barrios son nodos, los tendidos posibles son aristas no dirigidas con un costo de construcción y una capacidad máxima de transporte. Conectar todos los nodos al menor costo es exactamente el problema del árbol recubridor mínimo; verificar si las cañerías alcanzan para llevar agua desde la planta hasta los barrios es un problema de flujo máximo en una red con capacidades.

Decisiones de diseño principales:

- **Caso de estudio sintético:** 16 barrios en una cuadrícula de 4 × 4 más una planta (PLANTA), es decir 17 nodos y 26 tendidos posibles. Las coordenadas son ficticias.
- **Dos etapas separadas:** primero se diseña por costo (el costo no depende de la demanda) y después se valida la capacidad. No es una optimización conjunta de costo y abastecimiento; es una limitación asumida y declarada (sección 10).
- **Unidades:** el consumo se mide en m³/mes, la capacidad de las cañerías en m³/día y los costos en unidades convencionales (u. c.). Para comparar demanda y capacidad, el consumo mensual proyectado se divide por 30.

- **Dos casos de capacidades:** el caso Original usa las capacidades del dataset; el caso Progresivo cambia solo las capacidades para explorar cuándo la red deja de alcanzar (sección 5.3). Barrios, costos, modelo y red de menor costo son los mismos en ambos.
- **Arquitectura:** el backend (Python + FastAPI) contiene datos, modelo y algoritmos; el frontend (React + TypeScript + Vite) solo consume la API y presenta resultados. El navegador no calcula predicciones, árbol ni flujo.

Resumen del flujo de trabajo

El sistema sigue la cadena que propone el enunciado: problema → datos → modelo predictivo → grafo con pesos → algoritmo de grafos → aplicación web. Concretamente:

1. Un script genera el caso sintético (planta, 16 barrios, historial de 600 observaciones y 26 tendidos posibles).
2. Se entrena y evalúa un modelo de regresión que predice el consumo mensual a cinco años.
3. Ante cada simulación se modifica la tasa de crecimiento según el escenario, se predice la demanda de cada barrio y se convierte a m^3/día.
4. Kruskal calcula la red de menor costo sobre los costos de construcción.
5. Edmonds–Karp calcula cuánta de esa demanda puede abastecer la red, con las capacidades del caso elegido (Original o Progresivo), y marca los tendidos saturados.
6. La API devuelve todo en un JSON (red, flujos, balance total y abastecimiento por barrio) y el frontend lo dibuja.

Cumplimiento de la consigna

Requisito del enunciado	Dónde se cumple	Estado
Modelo de regresión con MAE, RMSE y R^2 sobre test separado	Sección 3	Cumplido: partición 80/20; modelos comparados por validación cruzada dentro del entrenamiento. El test se consultó durante el desarrollo (ver sección 3)
Usar la demanda proyectada como demanda de cada nodo	Secciones 4.2 y 5	Cumplido
Kruskal o Prim	Sección 4.1	Kruskal manual con Union-Find
Ford–Fulkerson o Edmonds–Karp, identificando aristas saturadas	Sección 4.2	Edmonds–Karp manual; saturación en diagnostico.py
Visualizar tendidos posibles con la red de menor costo resaltada	Sección 6.2, Figuras 2 a 4	Cumplido
Vista con flujo por arista, resaltando saturadas o cercanas al límite	Sección 6.3, Figuras 6 y 7	Cumplido (cercanía desde 90 %, saturación al 100 %)
Selector de escenario de crecimiento (+10 %, +30 %)	Sección 6.1, Figura 1	Cumplido (Base, +10 %, +30 %)
Dataset sintético con script y reglas declaradas	Sección 2	Cumplido
Documentar el uso de IA	Sección 9	Cumplido
Repositorio con historial incremental de commits	Repositorio	Cumplido
Demo end-to-end y README con instrucciones	README del repositorio	Cumplido: API en Render y frontend en Vercel
Funcionalidades adicionales: caso Original/Progresivo, suministro y déficit por barrio, comparación de escenarios	Secciones 5.3, 6.4 y 7	Incorporado

2. Dataset

Se usó un dataset sintético, generado por un script reproducible (`backend/app/data/generar_dataset.py`, semilla 42). El script produce cuatro CSV en `backend/data/raw/`:

Archivo	Contenido
planta.csv	Ubicación de la fuente de agua (1 fila).
zonas.csv	Los 16 barrios del caso, con las variables disponibles al predecir (sin la respuesta futura).
historial_consumo.csv	600 observaciones de entrenamiento, con la variable objetivo consumo_futuro.
aristas_candidatas.csv	26 tendidos posibles con costo y capacidad.

Reglas de generación (declaradas)

- **Población:** uniforme entre 200 y 1200 habitantes.
- **Tipo de zona:** residencial, comercial o industrial (aproximadamente 65 %, 25 % y 10 %).
- **Crecimiento poblacional anual estimado:** entre 0,5 % y 4 %.
- **Consumo histórico mensual:** 0,15 m³ por persona y día × 30 días × un factor según el tipo de zona (residencial 1,0; comercial 1,45; industrial 2,1), con una variación uniforme de ±12 %.
- **Consumo futuro (variable a predecir):** consumo histórico × (1 + crecimiento anual)^5, con una variación adicional de ±5 %.
- **Grafo:** cada barrio se conecta con sus vecinos horizontales y verticales y se agregan dos enlaces posibles desde la planta (26 aristas, grafo conexo). El costo es distancia en metros × 10 × un factor de dificultad entre 0,9 y 1,25. Los enlaces de la planta tienen capacidad 2500 m³/día; el resto toma valores de 750, 1200, 2000 o 3000 m³/día.

Estos parámetros son supuestos didácticos, no mediciones de una cooperativa real. El modelo va a descubrir las reglas que nosotros pusimos: es esperable y está bien, pero implica que las métricas miden cuánto aprende el modelo de un mundo simulado, no la precisión sobre consumo real.

Variables

Variable	Tipo	Rol
poblacion	numérica	feature
consumo_historico_mensual	numérica (m³/mes)	feature
crecimiento_poblacional_estimado	numérica (% anual)	feature
tipo_zona	categórica	feature (one-hot)
latitud, longitud, fila, columna	numéricas	espaciales; se evaluaron y se descartaron (sección 3)
consumo_futuro	numérica (m³/mes)	variable objetivo

Se agregaron fila y columna respecto de la tabla orientativa del enunciado para poder construir la cuadrícula. `zonas.csv` omite consumo_futuro a propósito: son las zonas para las que hay que predecirlo.

Estadísticas descriptivas del historial (600 observaciones)

Variable	Media	Desvío	Mínimo	Máximo
poblacion (habitantes)	719,72	275,96	201	1.200
consumo_historico_mensual (m³/mes)	3.972,14	2.063,58	870,90	12.143,17
crecimiento_poblacional_estimado (anual)	0,0235	0,0100	0,0050	0,0400
consumo_futuro (m³/mes)	4.468,19	2.339,97	1.012,39	14.847,70

Composición por tipo de zona: 397 residenciales (66,2 %), 136 comerciales (22,7 %) y 67 industriales (11,2 %), cercana a los pesos 65/25/10 del generador. El cociente consumo_futuro / consumo_historico_mensual tiene media 1,125 (mínimo 0,99; máximo 1,274): en promedio el consumo crece un 12,5 % en cinco años, con casos donde el ruido de ±5 % compensa un crecimiento bajo y el consumo queda apenas por debajo del actual.

Correlaciones de Pearson con la variable objetivo: consumo_historico_mensual 0,992; poblacion 0,754; crecimiento_poblacional_estimado 0,045. La correlación casi nula del crecimiento no significa que no importe: su efecto es multiplicativo (actúa como (1 + g)^5 sobre el consumo actual), y una correlación lineal simple no lo captura. Esto reaparece en la sección 3.

El consumo per cápita diario promedio que surge de los datos es de 0,150 m³ (residencial), 0,215 m³ (comercial) y 0,316 m³ (industrial), consistente con los factores 1,0, 1,45 y 2,1 aplicados a la base de 0,15 m³ por persona y día.

Los 16 barrios del caso de estudio (zonas.csv)

Son los barrios sobre los que se calcula la red. Se generan con las mismas reglas que el historial, pero sin la columna consumo_futuro: es lo que el modelo debe predecir. En la interfaz se muestran como “Barrio 01” a “Barrio 16”, equivalentes a Z01 a Z16.

Barrio	Fila, columna	Población	Tipo de zona	Consumo actual (m³/mes)	Crecimiento anual
Z01	0, 0	854	residencial	3.607,71	3,10 %
Z02	0, 1	342	comercial	2.441,59	2,87 %
Z03	0, 2	289	residencial	1.173,68	0,61 %
Z04	0, 3	438	residencial	1.828,54	0,59 %
Z05	1, 0	865	comercial	5.575,32	1,97 %
Z06	1, 1	484	comercial	3.389,89	0,52 %
Z07	1, 2	914	residencial	3.831,98	1,47 %
Z08	1, 3	981	residencial	3.987,23	0,82 %
Z09	2, 0	1.067	residencial	4.275,39	1,43 %
Z10	2, 1	670	residencial	2.927,11	3,91 %
Z11	2, 2	765	residencial	3.760,96	2,70 %
Z12	2, 3	570	residencial	2.285,41	2,97 %
Z13	3, 0	433	comercial	3.066,26	3,95 %
Z14	3, 1	1.087	residencial	5.050,79	1,47 %
Z15	3, 2	573	residencial	2.683,81	1,74 %
Z16	3, 3	918	industrial	8.902,31	2,77 %

La planta está en latitud −32,95 y longitud −60,65. Los barrios se ubican en una grilla con paso de 0,004° de latitud entre filas y 0,005° de longitud entre columnas (aproximadamente 445 m y 467 m), partiendo de la fila 0 a 0,004° al sur de la planta.

Tendidos candidatos

Son 26 aristas: 24 entre vecinos de la grilla (vertical y horizontal) y 2 desde la planta (a Z01 y a Z02). La distancia se calcula con la fórmula de haversine; el costo es distancia × 10 × un factor de dificultad aleatorio entre 0,9 y 1,25. La tabla completa, con la marca de las aristas que Kruskal incorporó a la red, está en el Anexo A y se puede consultar en la aplicación (Figura 4).

El costo depende de la distancia y de la dificultad: 25 de las 26 aristas cuestan entre 4.100 y 5.800 u. c.; solo el enlace PLANTA — Z02 (más largo, 644,6 m) sobresale con 7.995,33 u. c. La capacidad, en cambio, es independiente del costo (se sortea entre 750, 1200, 2000 y 3000): por eso un tendido barato puede ser un cuello de botella, que es exactamente lo que ocurre con Z01 — Z05.

Calidad y reproducibilidad de los datos

- El generador usa `random.Random(semilla)` con semilla 42 por defecto; admite `--semilla` y `--muestras` (mínimo 100 observaciones).
- Una prueba automática verifica que dos generaciones con la misma semilla produzcan los mismos CSV y que el grafo candidato sea conexo.
- Cada fila del historial es un barrio genérico (H0001…H0600) ubicado cíclicamente en la grilla; no es una serie temporal de un mismo barrio. Cada observación es independiente, lo que justifica una partición aleatoria en train y test.
- No hay valores faltantes porque los datos se generan completos. En datos reales habría que decidir cómo imputarlos.

3. Modelo predictivo

Objetivo: dado el perfil de un barrio, estimar su consumo mensual a cinco años (regresión).

Preparación de variables

tipo_zona se codifica con one-hot. Se verificó si convenía incluir las variables espaciales (latitud, longitud, fila, columna). El consumo no depende de ellas por construcción, y los resultados lo confirman: sin variables espaciales el error de validación cruzada es algo menor (MAE 142,65 → 141,80 m³/mes en regresión lineal y 194,37 → 186,66 en el árbol). Se descartaron para simplificar el modelo y evitar que aprenda correlaciones espurias producto del ruido.

Partición y validación

Se hizo una partición en dos etapas:

1. **Hold-out 80/20** con semilla 42: 480 observaciones de entrenamiento y 120 de test.
2. **Validación cruzada K-Fold (K = 5)** dentro del 80 % de entrenamiento, para comparar modelos y configuraciones de variables.

La selección del modelo y de las variables se basó en la validación cruzada. Sin embargo, la partición de test ya se había consultado durante el desarrollo (por ejemplo, para revisar errores y residuos), por lo que no corresponde presentarla como un conjunto reservado y evaluado una única vez. Las métricas de test se informan como referencia del desempeño y no como una estimación completamente independiente.

Modelo y configuración	MAE CV (m³/mes)	RMSE CV (m³/mes)	R^2 CV
Regresión lineal, sin espaciales	141,80	193,86	0,9930
Regresión lineal, con espaciales	142,65	194,68	0,9930
Árbol de decisión, sin espaciales	186,66	288,42	0,9846
Árbol de decisión, con espaciales	194,37	296,84	0,9837

Se eligió regresión lineal sin variables espaciales, que se reentrenó con todo el 80 % y se exportó a `backend/models/modelo_consumo.joblib`. Las variables finales son poblacion, consumo_historico_mensual, crecimiento_poblacional_estimado, tipo_zona_industrial y tipo_zona_residencial. Se compararon dos familias de modelos (regresión lineal y árbol de decisión); no se probaron otras ni se optimizaron hiperparámetros.

Resultados sobre el test (120 observaciones)

Métrica	Valor
MAE	130,11 m³/mes
RMSE	172,14 m³/mes
R^2	0,9940
MAPE	3,57 %

Análisis de errores por tipo de zona

Tipo de zona	N	Consumo prom. (m³/mes)	MAE	RMSE	MAPE	R^2
Residencial	82	3.557,7	106,83	133,63	3,79 %	0,9885
Comercial	28	5.201,4	128,09	157,71	2,62 %	0,9943
Industrial	10	8.637,4	326,67	373,51	4,48 %	0,9840

El error absoluto en zonas industriales es aproximadamente el triple que en las residenciales, pero eso se explica en buena parte por la escala: consumen más del doble (factor 2,1). El error relativo (MAPE) es parejo, entre 2,6 % y 4,5 %. Los peores casos de la muestra de residuos tienen errores cercanos al 8 % (por ejemplo, el nodo H0420, residencial: real 2967,02, predicho 3204,88; y el H0111, comercial: real 2306,38, predicho 2127,37), compatibles con el ruido de ±12 % y ±5 % que se inyectó al generar los datos.

Un R^2 tan alto no es mérito de la ingeniería del modelo, sino que refleja que los datos se generaron con una regla simple y poco ruidosa. La regresión lineal obtiene mejores métricas que el árbol de decisión. Una explicación plausible es que la relación es casi lineal en las variables elegidas y el árbol la aproxima con tramos constantes; también pueden influir el tamaño del entrenamiento (480 observaciones) y el ruido. No se hicieron experimentos (curvas de aprendizaje, ajuste de profundidad) que permitan confirmar la causa, por lo que se la presenta como hipótesis y no como un hecho.

Interpretación de los coeficientes

Variable	Coeficiente	Interpretación
consumo_historico_mensual	1,121	Cada m³/mes de consumo actual se traduce en unos 1,12 m³/mes futuros: coincide con (1 + g)^5 para una tasa de 2,35 % (la media), que da 1,123.
crecimiento_poblacional_estimado	22.639,5	Un punto porcentual más de crecimiento (0,01) suma unos 226 m³/mes. Es del orden de 5 × consumo medio × (1 + g)^4, la derivada de la fórmula real.
poblacion	0,069	Casi nulo: el consumo actual ya contiene la información de la población (correlación 0,77).
tipo_zona_industrial	17,3	Casi nulo por la misma razón: el factor del tipo de zona ya está en el consumo actual.
tipo_zona_residencial	−0,54	Ídem (la categoría de referencia es comercial).
Intercepto	−568,31	Compensa que el término lineal subestima el crecimiento compuesto.

El modelo, por tanto, aprendió la estructura de las reglas que se usaron para generar los datos: casi todo el poder predictivo está en el consumo actual y en la tasa de crecimiento. Es el comportamiento esperado de un modelo sobre datos sintéticos y conviene mencionarlo explícitamente en la defensa.

Residuos según la tasa de crecimiento

El residuo se define como valor real menos valor predicho (positivo: el modelo subestima). Sobre el test, el sesgo medio global es +6,53 m³/mes y el mayor error absoluto individual es 684,8 m³/mes.

Tasa de crecimiento	Observaciones	Residuo medio	Error absoluto medio
hasta 1 %	18	+57,7	159,2
1 % a 2 %	29	+17,4	117,7
2 % a 3 %	39	−11,4	97,3
3 % a 4 %	34	−9,3	162,9

El modelo tiende a subestimar con tasas bajas y a sobrestimar levemente en el rango medio. Es consistente con la idea de que una recta aproxima una relación multiplicativa, pero con pocas observaciones por tramo la lectura es orientativa y no concluyente.

Efecto del escenario sobre la predicción y riesgo de extrapolación

Los escenarios multiplican la tasa de crecimiento, no el consumo. Como la tasa actúa sobre cinco años, su efecto sobre la demanda es moderado: para g = 2 %, pasar a × 1,3 lleva el factor (1 + g)^5 de 1,104 a 1,137 (+3,0 %); para g = 4 %, de 1,217 a 1,288 (+5,9 %); para g = 0,5 %, solo +0,7 %. En el total de los 16 barrios la demanda sube de 2.167,63 a 2.242,09 m³/día, un +3,4 %.

El modelo se entrenó con tasas entre 0,5 % y 4 %. Con × 1,3 algunos barrios quedan fuera de ese rango (Z13 pasa de 3,95 % a 5,14 %), de modo que se está extrapolando. Como control, se comparó el total predicho con el que daría la fórmula sin ruido, consumo actual × (1 + g)^5 (verificación hecha al redactar el informe, reproducible con esa fórmula sobre zonas.csv):

Multiplicador	Tasa máxima entre los 16 barrios	Demanda total predicha (m³/día)	Demanda con fórmula sin ruido (m³/día)	Diferencia
× 1,0	3,95 %	2.167,6	2.179,1	−0,53 %
× 1,1	4,34 %	2.192,4	2.202,3	−0,45 %
× 1,3	5,14 %	2.242,1	2.249,5	−0,33 %

Para los escenarios del enunciado, la diferencia en el total es menor al 0,6 %, despreciable frente al déficit. La API acepta multiplicadores hasta 5, donde la extrapolación sí sería riesgosa; la validez del modelo se limita a escenarios cercanos al rango de entrenamiento.

Limitaciones de la evaluación

- Partición aleatoria única (semilla 42): no se repitió con varias semillas ni se calcularon intervalos de confianza para las métricas.
- El conjunto de test se consultó durante el desarrollo; sus métricas no son una estimación independiente.
- Los hiperparámetros del árbol de decisión no se optimizaron con búsqueda en grilla.
- El test tiene solo 10 observaciones industriales: las métricas de ese grupo son menos estables.
- Las observaciones son independientes y sintéticas; no hay efecto temporal ni espacial que evaluar.

4. Algoritmos de grafos

Ambos algoritmos están implementados manualmente. NetworkX se usa para representar los grafos y como referencia para contrastar resultados, no para resolverlos.

4.1 Red de menor costo: Kruskal

Archivo: `backend/app/grafos/mst.py`, con la estructura UnionFind (compresión de caminos y unión por rango) en `union_find.py`.

1. Se ordenan las 26 aristas candidatas por costo creciente.
2. Se recorren en ese orden; una arista se acepta si une dos componentes distintas (union devuelve verdadero), y se descarta si cerraría un ciclo.
3. Al terminar se verifica que todos los nodos esperados (planta y 16 barrios) estén en una sola componente; si no, se rechaza el resultado en lugar de presentar un bosque como red completa (la API responde HTTP 503).

Complejidad: O(E log E) por el ordenamiento. El resultado son 16 aristas (V − 1) con un costo total de 73.066,05 u. c. Como control, el costo se compara con `networkx.minimum_spanning_tree` (tolerancia 0,1). Con costos empatados puede haber árboles distintos de igual costo, por eso se compara el costo y no el conjunto de aristas.

4.2 Verificación de capacidad: Edmonds–Karp

Archivo: `backend/app/grafos/flujo.py`. Construcción de la red de flujo sobre el árbol obtenido:

- **Fuente:** PLANTA.

- **Aristas del árbol:** cada tendido se modela como dos arcos dirigidos (uno por sentido) con la capacidad máxima del tendido, en m³/día. La capacidad depende del caso elegido (Original o Progresivo, sección 5.3).
- **Sumidero ficticio (SUPER_SUMIDERO):** cada barrio se conecta a él con un arco cuya capacidad es exactamente su demanda diaria proyectada. Así el flujo máximo representa cuánta agua puede entregar la red a todos los barrios a la vez, y el flujo que llega al sumidero por el arco de un barrio es el suministro efectivo de ese barrio.

Algoritmo: se buscan caminos de aumento con BFS (caminos más cortos en cantidad de aristas), se envía el cuello de botella del camino y se actualiza el flujo residual en ambos sentidos, hasta que no haya más caminos. Complejidad $O(V \cdot E^2)$. El flujo total se contrasta con `networkx.maximum_flow` (tolerancia 0,1); las asignaciones por arista pueden diferir porque puede haber varios flujos máximos.

Diagnóstico (`diagnostico.py`): para cada tendido seleccionado se calcula flujo, utilización (flujo / capacidad) y estado. Se considera saturado si el flujo alcanza la capacidad, y cerca del límite si la utilización es ≥ 90 %. El déficit es demanda total − caudal abastecido; el déficit de cada barrio es su demanda − su suministro.

4.3 Pseudocódigo

```
KRUSKAL(nodos, aristas):
  uf ← UnionFind(nodos); arbol ← ∅; costo ← 0
  para cada arista (u, v, costo_uv, cap) de aristas ordenadas por costo:
    si uf.union(u, v):   # u y v estaban en componentes distintas
      agregar (u, v) a arbol; costo ← costo + costo_uv
  si no todos los nodos tienen la misma raíz: error (red no conexa)
  devolver arbol, costo

EDMONDS-KARP(nodos, arcos, s, t):
  capacidad[u][v] ← cap de cada arco; capacidad[v][u] ← 0 si no existe
  flujo ← 0 en todo
  mientras BFS encuentre un camino s → t con capacidad residual > 0:
    delta ← mínimo de (capacidad − flujo) a lo largo del camino
    para cada arco (u, v) del camino:
      flujo[u][v] += delta; flujo[v][u] −= delta
    flujo_maximo += delta
  devolver flujo_maximo, flujo
```

4.4 Traza de Kruskal sobre el caso

Las 26 aristas ordenadas por costo, con la decisión que toma el algoritmo y el costo acumulado de la red. Se aceptan 16 aristas (V − 1, con V = 17 nodos) y se rechazan 10 por cerrar un ciclo.

Orden	Tendido	Costo (u. c.)	Capacidad (m³/día)	Decisión	Costo acumulado
1	Z01 — Z05	4.147,83	1.200	Acepta	4.147,83
2	Z09 — Z13	4.244,95	1.200	Acepta	8.392,78
3	Z05 — Z06	4.320,69	750	Acepta	12.713,47
4	Z13 — Z14	4.345,75	2.000	Acepta	17.059,22
5	Z08 — Z12	4.366,69	1.200	Acepta	21.425,91
6	Z12 — Z16	4.415,56	750	Acepta	25.841,47
7	Z09 — Z10	4.420,55	750	Acepta	30.262,02
8	Z01 — Z02	4.431,17	2.000	Acepta	34.693,19
9	Z07 — Z11	4.498,15	2.000	Acepta	39.191,34
10	Z15 — Z16	4.549,18	2.000	Acepta	43.740,52
11	Z06 — Z07	4.585,10	2.000	Acepta	48.325,62
12	Z10 — Z14	4.635,85	2.000	Rechaza (ciclo)	48.325,62
13	Z05 — Z09	4.680,17	1.200	Acepta	53.005,79
14	Z11 — Z15	4.706,53	3.000	Acepta	57.712,32
15	Z03 — Z07	4.907,12	1.200	Acepta	62.619,44

Orden	Tendido	Costo (u. c.)	Capacidad (m³/día)	Decisión	Costo acumulado
16	Z10 — Z11	4.932,32	2.000	Rechaza (ciclo)	62.619,44
17	Z03 — Z04	4.947,29	750	Acepta	67.566,73
18	Z04 — Z08	4.994,99	3.000	Rechaza (ciclo)	67.566,73
19	Z02 — Z06	5.094,05	2.000	Rechaza (ciclo)	67.566,73
20	Z06 — Z10	5.327,66	3.000	Rechaza (ciclo)	67.566,73
21	Z07 — Z08	5.342,63	2.000	Rechaza (ciclo)	67.566,73
22	PLANTA — Z01	5.499,32	2.500	Acepta	73.066,05
23	Z14 — Z15	5.547,30	750	Rechaza (ciclo)	73.066,05
24	Z02 — Z03	5.663,82	3.000	Rechaza (ciclo)	73.066,05
25	Z11 — Z12	5.773,26	1.200	Rechaza (ciclo)	73.066,05
26	PLANTA — Z02	7.995,33	2.500	Rechaza (ciclo)	73.066,05

Observaciones sobre la traza:

- Z10 — Z14 se rechaza en el paso 12 porque Z10 y Z14 ya estaban conectados por Z10 — Z09 — Z13 — Z14. Todos los rechazos responden a la misma situación: los dos extremos ya pertenecen a la misma componente.
- La planta se incorpora al final (paso 22) porque sus dos enlaces son los más caros. Entra por PLANTA — Z01 (5.499,32) y no por PLANTA — Z02 (7.995,33), que es 2.496 u. c. más cara.
- Kruskal no mira la capacidad. Eligió Z01 — Z05 (1.200 m³/día) por ser la arista más barata de toda la red, y con eso quedó definida como única entrada a la mayor parte de la zona. Esa decisión, razonable por costo, es la causa del déficit en el caso Original.

4.5 La red resultante

Tendido	Costo (u. c.)	Capacidad caso Original (m³/día)
PLANTA — Z01	5.499,32	2.500
Z01 — Z02	4.431,17	2.000
Z01 — Z05	4.147,83	1.200
Z03 — Z04	4.947,29	750
Z03 — Z07	4.907,12	1.200
Z05 — Z06	4.320,69	750
Z05 — Z09	4.680,17	1.200
Z06 — Z07	4.585,10	2.000
Z07 — Z11	4.498,15	2.000
Z08 — Z12	4.366,69	1.200
Z09 — Z10	4.420,55	750
Z09 — Z13	4.244,95	1.200
Z11 — Z15	4.706,53	3.000
Z12 — Z16	4.415,56	750
Z13 — Z14	4.345,75	2.000
Z15 — Z16	4.549,18	2.000
Total (16 tendidos)	**73.066,05**	

Como es un árbol, entre la planta y cada barrio hay exactamente un camino. Por eso el flujo máximo en este caso queda determinado por cortes de una sola arista, que es lo que hace tan visible el cuello de botella.

4.6 Ejemplo resuelto de Edmonds–Karp

Es el caso controlado que usan las pruebas: PLANTA — A con capacidad 70, A — B con capacidad 35, demanda de A = 26 y de B = 39 (demanda total 65). Se agregan los arcos A → S (capacidad 26) y B → S (capacidad 39).

1. **Primer camino de aumento (BFS):** PLANTA → A → S, el más corto en cantidad de aristas. Cuello de botella: min(70, 26) = 26. Flujo total: 26.
2. **Segundo camino:** PLANTA → A → B → S. Capacidades residuales: 70 − 26 = 44, 35 y 39; cuello de botella 35. Flujo total: 61.
3. **Tercer intento:** el BFS ya no alcanza S. A — B está saturado y el residuo de B → S (4 m³/día) es inalcanzable desde la planta.

Resultado: flujo máximo 61, déficit 65 − 61 = 4, con A — B al 100 %. Es el mismo valor que el caso “Insuficiente” del contrato de la API. Con demandas A = 20 y B = 30 el flujo es 50 y el déficit es 0.

4.7 Corrección, complejidad y detalles de implementación

- **Kruskal:** es correcto por la propiedad de corte: la arista de menor costo que cruza un corte pertenece a algún árbol recubridor mínimo, y rechazar una arista que cierra un ciclo no puede empeorar el costo. Con Union-Find con compresión de caminos y unión por rango, el costo es O(E log E), dominado por el ordenamiento. Con E = 26 es instantáneo.
- **Edmonds–Karp:** al elegir siempre el camino más corto en aristas, el número de aumentos es O(V·E) y el total es $O(V \cdot E^2)$. Termina incluso con capacidades reales (no enteras), a diferencia de Ford–Fulkerson con DFS arbitrario. El flujo es antisimétrico (flujo[v][u] −= delta), lo que permite cancelar flujo previo, y los arcos inversos se inicializan con capacidad 0.
- **Corrección del resultado:** por el teorema de flujo máximo y corte mínimo, el valor máximo es igual a la capacidad de un corte mínimo. En el escenario +30 % del caso Original el corte mínimo es {PLANTA, Z01, Z02} contra el resto, con capacidad 1.200 + 148,19 + 101,22 = 1.449,41, igual al caudal abastecido.
- **Tendidos no dirigidos:** cada tendido se modela como dos arcos de igual capacidad. Para informar un tendido se toma el mayor de los dos flujos y se acota por debajo en cero (flujo neto). Un tendido se marca saturado si el flujo alcanza su capacidad con tolerancia de 10^{-5} y se reporta como crítico desde el 90 % de utilización.
- **Verificación cruzada:** el código compara el costo de Kruskal y el flujo de Edmonds–Karp con NetworkX mediante assert. Esas verificaciones no corren si Python se ejecuta con la opción -O; es una limitación menor, pero conviene saberlo.
- **Qué es propio y qué es de la librería:** Kruskal, Union-Find y Edmonds–Karp son propios. NetworkX solo guarda el árbol resultante y sirve de referencia.

5. Integración modelo + grafos

5.1 Pipeline

El pipeline (`backend/app/servicios/pipeline.py`) ejecuta, en orden y ante cada simulación:

1. **Escenario:** el usuario elige Base, +10 % o +30 %, que multiplica la tasa de crecimiento poblacional de cada barrio por 1, 1,1 o 1,3. No aumenta directamente el consumo en ese porcentaje.
2. **Predicción:** el modelo estima el consumo mensual de cada barrio con la tasa modificada.
3. **Conversión de unidades:** consumo mensual ÷ 30 = demanda diaria (m³/día).
4. **Kruskal:** red de menor costo sobre los costos fijos.
5. **Caso de capacidades:** se aplican las capacidades del caso elegido (Original o Progresivo) a los tendidos del árbol.
6. **Edmonds–Karp:** la demanda diaria predicha de cada barrio fija la capacidad de su arco hacia el sumidero.
7. **Respuesta JSON:** 17 nodos, 26 candidatos, árbol seleccionado, los 16 flujos, el balance total (resumen_capacidad) y el abastecimiento de cada barrio.

Qué salida del modelo entra al algoritmo y por qué es razonable. La predicción de consumo de cada barrio es la demanda del nodo y determina la capacidad de su arco al sumidero; si el modelo predice más consumo, el flujo máximo, los tramos saturados y el déficit pueden cambiar. Es razonable porque el agua que la red debe entregar a un barrio es precisamente su demanda proyectada, y el flujo máximo responde la pregunta de la cooperativa: ¿cuánto de esa demanda llega?

Qué no cambia. Los costos son fijos y no dependen de la demanda ni del caso de capacidades; por eso el árbol de menor costo es el mismo en todos los escenarios y casos. Lo que cambia es el diagnóstico de capacidad. Es coherente con "diseñar primero por costo, validar después".

5.2 Contrato de la API

La API (FastAPI, documentación interactiva en /docs) se expone con CORS abierto y sin autenticación.

Método y ruta	Función
POST /api/simular-red	Endpoint principal: ejecuta el pipeline completo para un escenario y un caso de capacidades.
GET /api/red/nodos	Lista los 17 nodos (planta y barrios).
GET /api/red/conexiones	Lista las 26 conexiones candidatas.
GET /api/red/mst	Devuelve la red de menor costo y su costo total (Kruskal).
GET /api/modelo/status	Verifica que el modelo exista y devuelve su clase y variables esperadas.
GET /api/modelo/metricas	Métricas del modelo sobre el test (null si el archivo falta o es inválido).
POST /api/modelo/predecir	Predicción aislada del consumo futuro de una zona (m³/mes y m³/día).

Solicitud de /api/simular-red: `{"escenario_crecimiento": 1.3}`. El valor debe ser mayor que 0 y no superar 5; los presets de la página son 1,0, 1,1 y 1,3. La solicitud indica además el caso de capacidades elegido en el campo `caso_red` (`"original"` o `"progresivo"`; si se omite, vale `"original"`), por ejemplo `{"caso_red": "progresivo", "escenario_crecimiento": 1.1}`. Un caso desconocido se rechaza con HTTP 422. La respuesta repite `caso_red` y `escenario_crecimiento` para confirmarlos, y cada barrio de `nodos` incluye, junto con `demanda_diaria_proyectada`, los campos `caudal_abastecido_m3_dia` (suministro efectivo del barrio) y `deficit_m3_dia`. La planta no los informa porque es la fuente.

Respuesta (resumida, escenario +30 %, caso Original):

```
{
  "nodos": [ {"id": "PLANTA", "latitud": -32.95, "longitud": -60.65,
              "tipo_zona": "PLANTA", "demanda_diaria_proyectada": 0.0},
             {"id": "Z01", ..., "tipo_zona": "residencial",
              "demanda_diaria_proyectada": 148.19}, ... ],
  "tendidos_mst": [ {"origen": "PLANTA", "destino": "Z01", "costo": 5499.32,
                     "capacidad_maxima": 2500.0}, ... ],
  "tendidos_candidatos": [ ...26 tendidos... ],
  "flujos": [ {"origen": "Z05", "destino": "Z01", "flujo_actual": 1200.0,
               "capacidad_maxima": 1200.0, "saturada": true}, ... ],
  "resumen_capacidad": {"demanda_total_m3_dia": 2242.09,
                        "caudal_abastecido_m3_dia": 1449.41,
                        "deficit_m3_dia": 792.68},
  "metricas_ml": {"mae": 130.11, "rmse": 172.14, "r2": 0.994}
}
```

Códigos de error: 422 si el cuerpo no cumple el esquema (por ejemplo, un multiplicador negativo o mayor que 5, o un tipo de zona desconocido en la predicción individual); 503 si la red candidata no permite conectar todos los nodos; 500 ante fallas inesperadas.

Reglas del intercambio que el frontend valida antes de mostrar resultados:

- Todos los tendidos del árbol deben estar entre los candidatos con los mismos costos y capacidades del caso informado.
- La API debe confirmar el caso de capacidades pedido; si no lo hace (por ejemplo, un backend desactualizado), la interfaz no muestra resultados de otro caso (Figura 12).

- Se informan los flujos de todos los tendidos del árbol, incluido el flujo cero; el flujo es no negativo y no supera la capacidad.
- resumen_capacidad debe ser consistente: la demanda total coincide con la suma de las demandas de los barrios, el abastecido no supera la demanda y el déficit es su diferencia (tolerancia de redondeo).
- La página no reconstruye el abastecimiento sumando caudales de las cañerías, porque contaría el mismo agua más de una vez; usa el total que calcula el backend.
- Si falta el resumen o alguna métrica, la interfaz lo declara como diagnóstico incompleto y no afirma que el agua alcance.

5.3 Casos de capacidades: Original y Progresivo

Además del escenario de crecimiento, la interfaz permite elegir el caso de capacidades (Figura 1). El caso cambia únicamente las capacidades de los tendidos: conserva barrios, costos y modelo, de modo que la demanda proyectada y la red de menor costo son idénticas en ambos.

- **Original:** capacidades del archivo aristas_candidatas.csv (750 a 3.000 m³/día; 2.500 en los enlaces de la planta). Es el caso de las secciones 4 a 7, donde el cuello de botella es el tramo interno Z01 — Z05.
- **Progresivo ("límite de crecimiento"):** está pensado para explorar cuándo la red deja de alcanzar. En la corrida de +10 % (Figuras 8 a 11), los 15 tendidos del árbol distintos del de entrada tienen capacidad 2.500 m³/día y PLANTA — Z01 tiene 2.195 m³/día. Como ninguna cañería interna limita el flujo, el límite pasa a ser la entrada desde la planta. La regla es una constante y no depende del escenario ni de la demanda: `generar_caso_progresivo.py` copia `aristas_candidatas.csv`, asigna 2.195 m³/día a PLANTA — Z01 y 2.500 m³/día a todos los demás tendidos candidatos, y conserva distancias y costos. El valor 2.195 es una elección sintética, apenas por encima de la demanda del escenario +10 % (2.192,45 m³/día) y por debajo de la del +30 %, para mostrar una transición de cobertura; no es un dimensionamiento hidráulico.

Con el caso Progresivo y +10 %, la demanda total (2.192,42 m³/día) queda por debajo de la capacidad de entrada y la red la cubre por completo, con déficit 0. El tramo PLANTA — Z01 opera al 99,9 % (menos de 3 m³/día de margen) y por eso se informa como "cerca del límite" y cuenta como tramo crítico. Como la capacidad de entrada es fija entre escenarios, con más demanda aparece déficit. Al ejecutar el pipeline con el caso Progresivo: en Base (2.167,62 m³/día) la red cubre todo y la entrada opera al 98,8 %; en +30 % (2.242,09 m³/día) la entrada se satura en 2.195 m³/día y queda un déficit de 47,09 m³/día.

La utilidad del caso Progresivo es comparar dos tipos de limitación: en el caso Original el déficit lo produce un tendido barato y angosto en medio de la red; en el Progresivo, la red interna es holgada y lo que se agota es la capacidad de entrega de la planta.

5.4 Decisiones de integración

- **Separación de responsabilidades:** el modelo, Kruskal y Edmonds–Karp viven únicamente en el backend. El frontend valida el contrato y presenta; no hay resultados simulados ni modo demostración que se haga pasar por el modelo.
- **Carga única:** el modelo y los CSV se cargan una vez al iniciar el servidor, por lo que cada simulación solo calcula predicciones y grafos. Si se regeneran los datos o se reentrena el modelo, hay que reiniciar la API.
- **Consistencia de las predicciones:** en inferencia las variables categóricas se codifican con todas las categorías y se alinean con las columnas del modelo entrenado, de modo que predecir una zona sola o dentro del lote da el mismo valor (corrección de un error de la IA, sección 9).
- **Despliegue:** la API está publicada en Render y el frontend en Vercel; también puede ejecutarse local contra la API publicada o contra el backend local mediante VITE_API_URL. La primera respuesta tras un período de inactividad de la API puede demorar.

6. Aplicación web

La interfaz (React, TypeScript estricto y Vite, con CSS propio y gráfico SVG) consume `POST /api/simular-red`. Tiene tres pestañas: **Tendidos**, **Capacidad** y **Efectos** (esta última es puramente visual y no interviene en los cálculos). Las capturas de las secciones 6.2 y 6.3 corresponden al escenario +30 % del caso Original.

6.1 Selector de escenario y de caso

En la parte superior se elige el caso de capacidades (Original o Progresivo) y el aumento de la tasa de crecimiento (Base, +10 % o +30 %), y el botón “Actualizar simulación” ejecuta la simulación en el backend. El texto aclaratorio recuerda que +30 % multiplica la tasa de cada barrio por 1,3 y no aumenta directamente el consumo un 30 %, y que el caso cambia capacidades pero conserva barrios, costos y modelo (Figura 1).

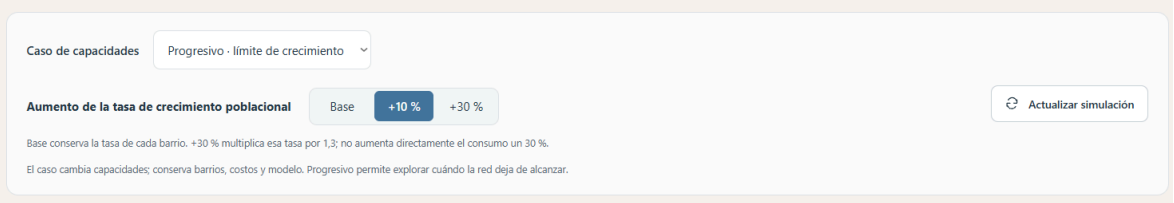


Figura 1. Selector de caso de capacidades (Progresivo · límite de crecimiento) y de aumento de la tasa de crecimiento (+10 %), con el botón para actualizar la simulación.

6.2 Pestaña Tendidos

La vista general (Figura 2) muestra el horizonte de análisis (5 años) y cuatro indicadores del resultado: costo del tendido (73.066,05 u. c.), demanda proyectada (2.242,1 m³/día), conexiones del diseño (16 tendidos para 16 barrios) y tramos críticos informados (1, es decir, con utilización de al menos 90 %). La tarjeta lateral aclara que se trata de datos sintéticos. Esta captura es anterior al selector de caso y corresponde al caso Original.

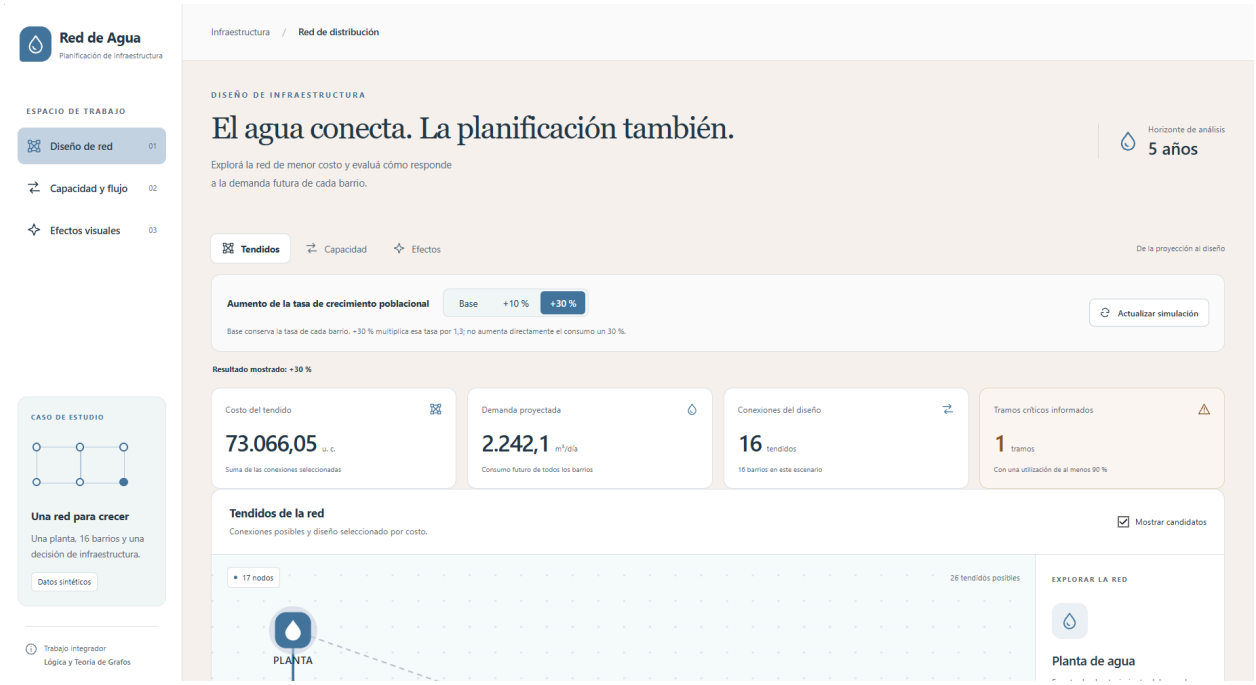


Figura 2. Vista general de la pestaña Tendidos (escenario +30 %, caso Original): selector e indicadores de costo, demanda, conexiones y tramos críticos.

El esquema de la grilla 4 × 4 con la planta (Figura 3) dibuja con línea continua la red seleccionada por Kruskal y con línea punteada los tendidos posibles, que se pueden ocultar con “Mostrar candidatos”. Al seleccionar un tendido, por ejemplo Z05 — Z06, el panel lateral muestra su costo (4.320,69 u. c.), capacidad máxima (750 m³/día), flujo informado (379,8 m³/día) y utilización (50,6 %). El esquema no está a escala de obra.

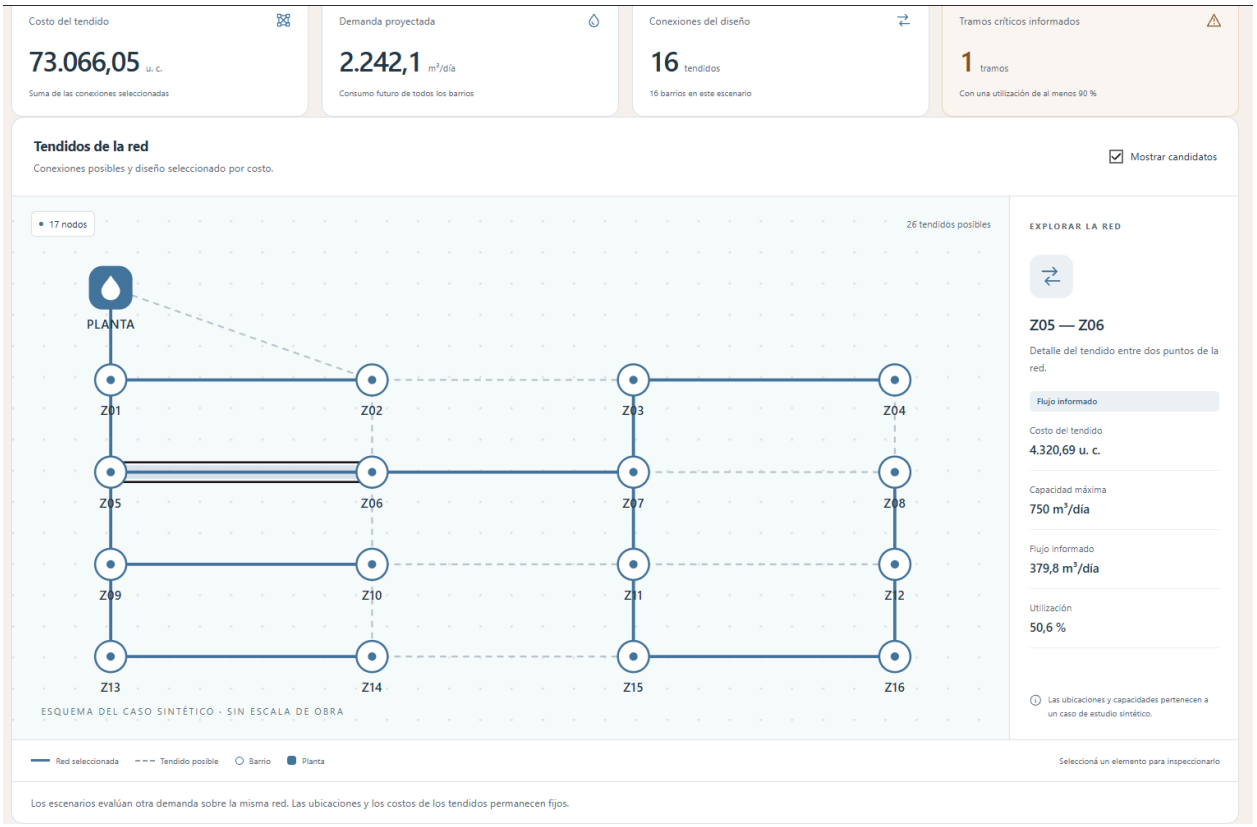
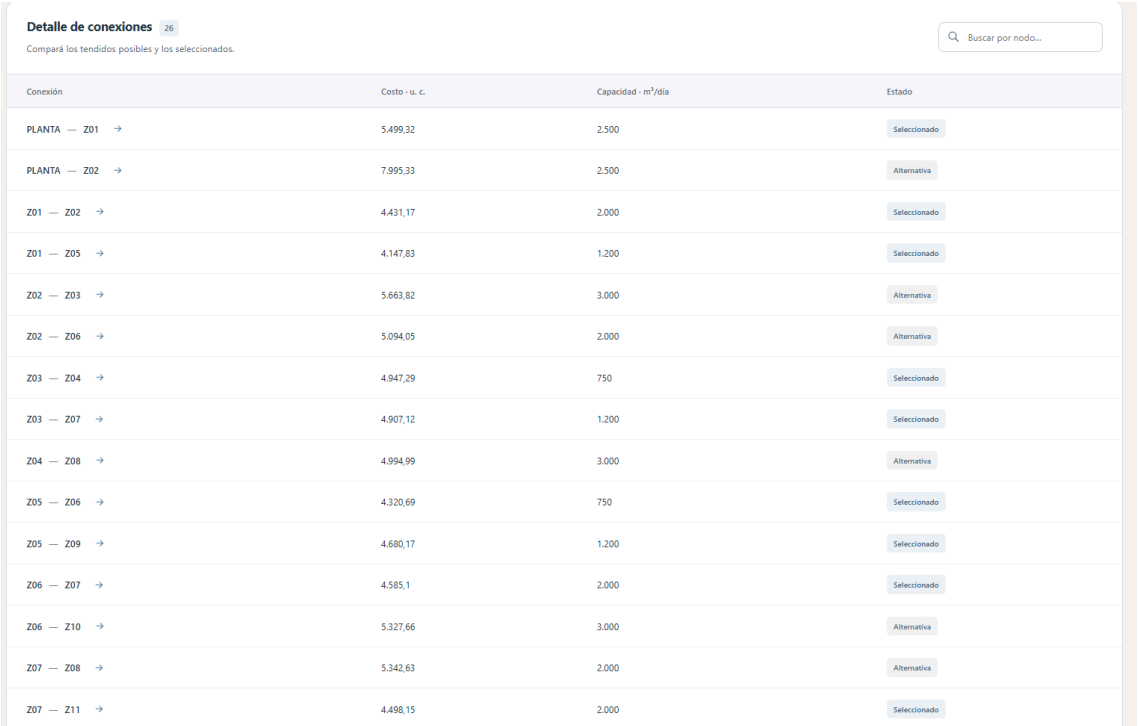


Figura 3. Red de menor costo (línea continua) sobre los tendidos posibles (punteada). El panel lateral muestra el detalle de la conexión Z05 — Z06.

La tabla de conexiones (Figura 4) lista los 26 tendidos posibles con costo, capacidad y estado (Seleccionado o Alternativa) y permite buscar por nodo. Sirve para comparar, por ejemplo, PLANTA — Z01 (seleccionado, 5.499,32) con PLANTA — Z02 (alternativa, 7.995,33). La captura muestra las primeras 15 filas.



Detalle de conexiones 26

Compará los tendidos posibles y los seleccionados.

Buscar por nodo...

Conexión	Costo · u. c.	Capacidad · m³/día	Estado
PLANTA — Z01	5.499,32	2.500	Seleccionado
PLANTA — Z02	7.995,33	2.500	Alternativa
Z01 — Z02	4.431,17	2.000	Seleccionado
Z01 — Z05	4.147,83	1.200	Seleccionado
Z02 — Z03	5.663,82	3.000	Alternativa
Z02 — Z06	5.094,05	2.000	Alternativa
Z03 — Z04	4.947,29	750	Seleccionado
Z03 — Z07	4.907,12	1.200	Seleccionado
Z04 — Z08	4.994,99	3.000	Alternativa
Z05 — Z06	4.320,69	750	Seleccionado
Z05 — Z09	4.680,17	1.200	Seleccionado
Z06 — Z07	4.585,1	2.000	Seleccionado
Z06 — Z10	5.327,66	3.000	Alternativa
Z07 — Z08	5.342,63	2.000	Alternativa
Z07 — Z11	4.498,15	2.000	Seleccionado

Figura 4. Detalle de las conexiones candidatas con costo, capacidad y estado (Seleccionado / Alternativa).

6.3 Pestaña Capacidad

El panel de diagnóstico de abastecimiento (Figura 5) resume el balance del backend para el escenario mostrado. En el caso Original con +30 % indica “La red no cubre toda la demanda”: demanda total 2.242,09 m³/día, caudal abastecido 1.449,41 y déficit 792,68 (en rojo). Estos tres números provienen de resumen_capacidad y no se recalculan en el navegador.

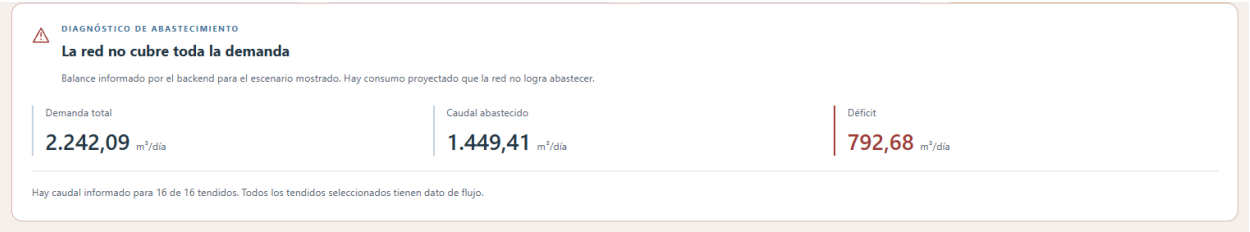


Figura 5. Diagnóstico de abastecimiento (+30 %, caso Original): la red no cubre toda la demanda.

El gráfico de flujo y capacidad (Figura 6) etiqueta cada tendido seleccionado con su caudal y su porcentaje de utilización. El tramo Z01 — Z05 aparece en rojo, saturado (1.200 de 1.200 m³/día). Se leen también PLANTA — Z01 con 1.449,4 m³/día (58 %), Z05 — Z09 con 609,5 (50,8 %), Z05 — Z06 con 379,8 (50,6 %) y Z09 — Z13 con 322 (26,8 %); varios tendidos del lado derecho muestran 0. La leyenda distingue azul (red seleccionada), naranja (cerca del límite), rojo (saturado) y línea punteada (flujo no informado).

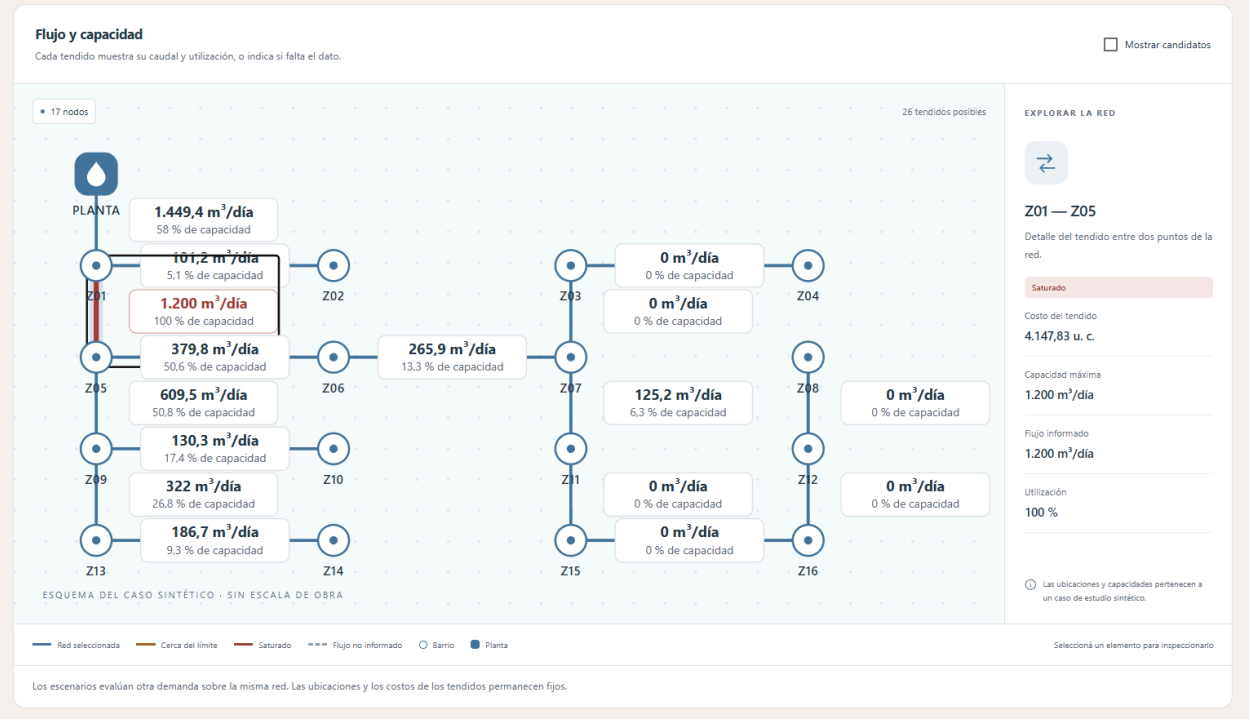
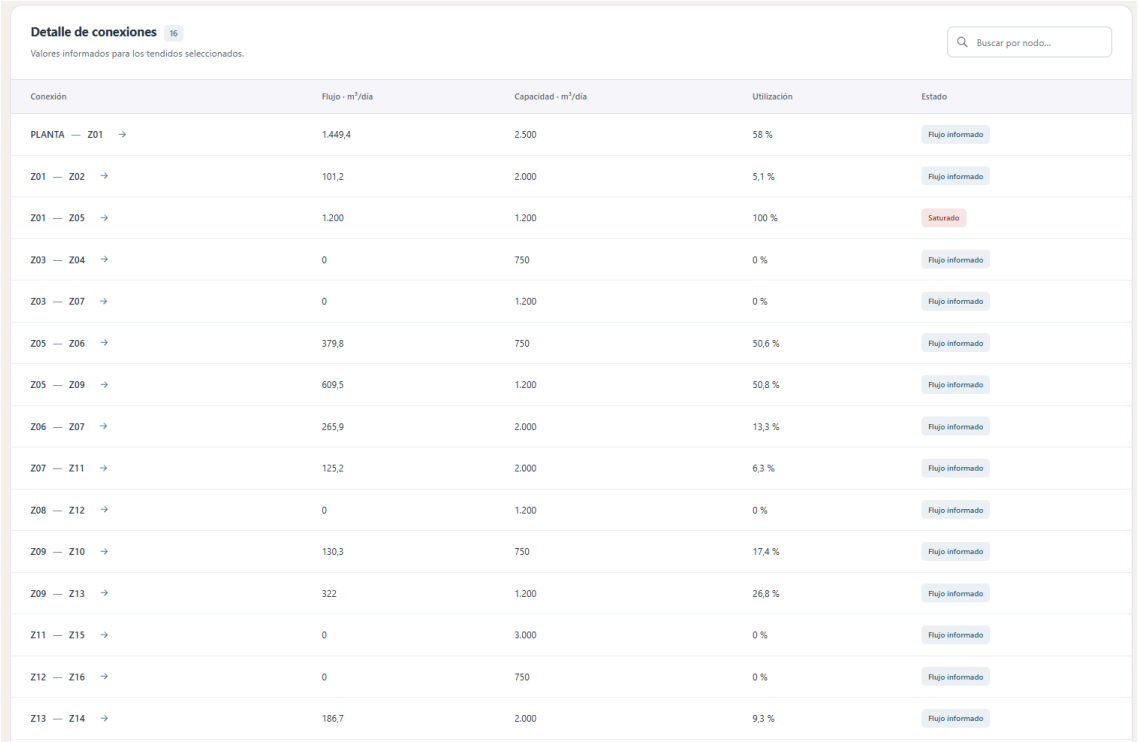


Figura 6. Pestaña Capacidad (+30 %, caso Original): flujo y utilización por tendido; el tramo Z01 — Z05 está saturado (en rojo).

La tabla de flujos (Figura 7) reúne los 16 tendidos de la red con flujo, capacidad, utilización y estado (Flujo informado o Saturado). Distingue el flujo cero real, que se muestra como 0, del dato ausente, que se muestra como “Sin dato”.



Detalle de conexiones 16

Valores informados para los tendidos seleccionados.

Buscar por nodo...

Conexión	Flujo · m³/día	Capacidad · m³/día	Utilización	Estado
PLANTA — Z01 →	1.449,4	2.500	58 %	Flujo informado
Z01 — Z02 →	101,2	2.000	5,1 %	Flujo informado
Z01 — Z05 →	1.200	1.200	100 %	Saturado
Z03 — Z04 →	0	750	0 %	Flujo informado
Z03 — Z07 →	0	1.200	0 %	Flujo informado
Z05 — Z06 →	379,8	750	50,6 %	Flujo informado
Z05 — Z09 →	609,5	1.200	50,8 %	Flujo informado
Z06 — Z07 →	265,9	2.000	13,3 %	Flujo informado
Z07 — Z11 →	125,2	2.000	6,3 %	Flujo informado
Z08 — Z12 →	0	1.200	0 %	Flujo informado
Z09 — Z10 →	130,3	750	17,4 %	Flujo informado
Z09 — Z13 →	322	1.200	26,8 %	Flujo informado
Z11 — Z15 →	0	3.000	0 %	Flujo informado
Z12 — Z16 →	0	750	0 %	Flujo informado
Z13 — Z14 →	186,7	2.000	9,3 %	Flujo informado

Figura 7. Flujos informados para los tendidos seleccionados (+30 %, caso Original).

6.4 Caso Progresivo y abastecimiento por barrio

Con el caso Progresivo y +10 %, el esquema de tendidos (Figura 8) es el mismo, porque el árbol no cambia, pero el panel lateral muestra otras capacidades. PLANTA — Z01 tiene capacidad máxima de 2.195 m³/día, flujo informado de 2.192,5 m³/día y utilización de 99,9 %, estado “Cerca del límite”.

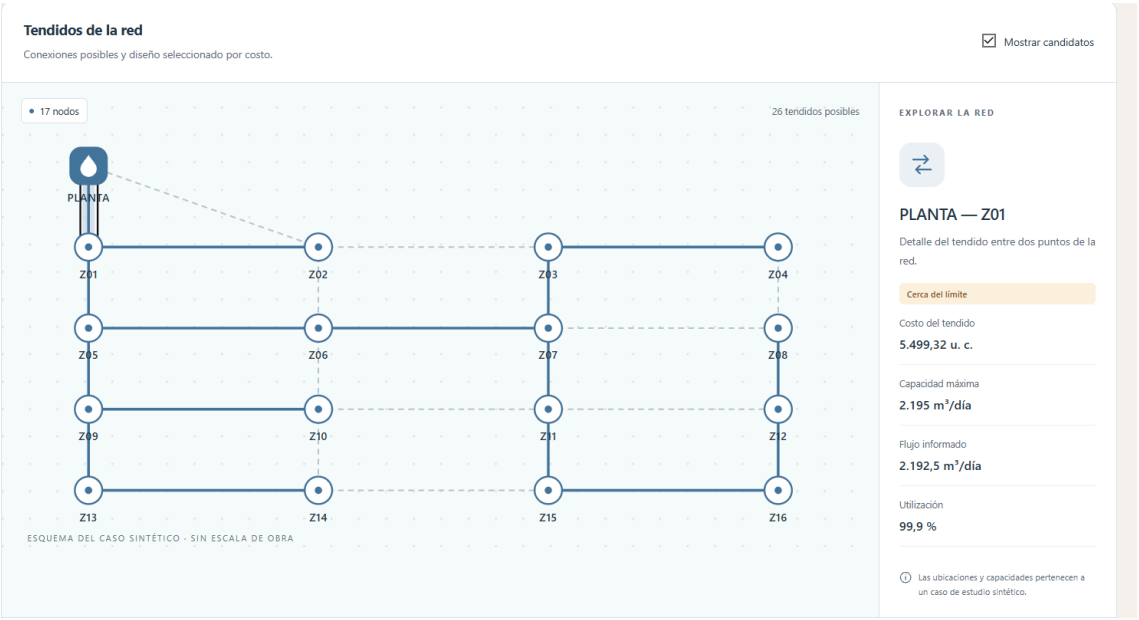


Figura 8. Caso Progresivo, +10 %: detalle del tendido PLANTA — Z01 (2.195 m³/día de capacidad, 99,9 % de utilización, cerca del límite).

El diagnóstico (Figura 9) pasa a “La red cubre la demanda proyectada”: demanda total y caudal abastecido de 2.192,42 m³/día y déficit 0. En el gráfico de flujo, la entrada de la planta aparece en naranja; el tramo más cargado de los internos es Z01 — Z05, al 78,1 % de su capacidad de 2.500 m³/día.

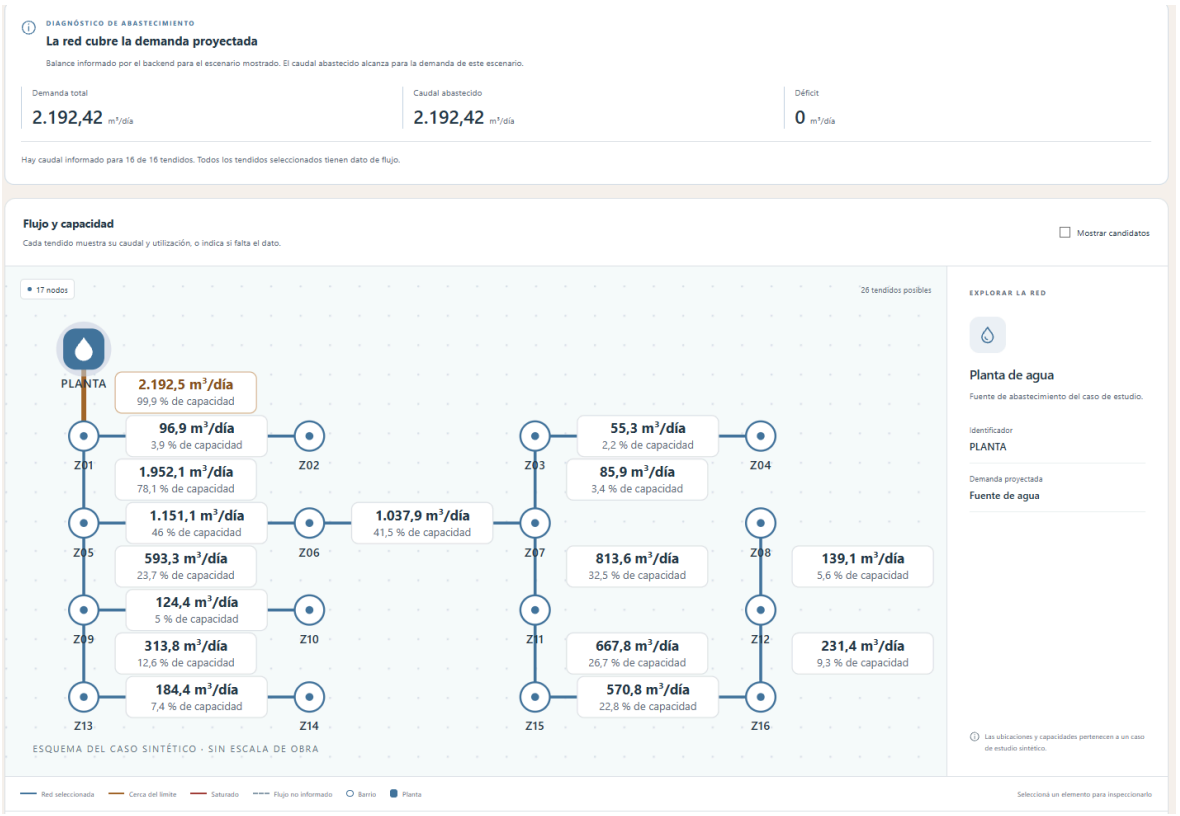
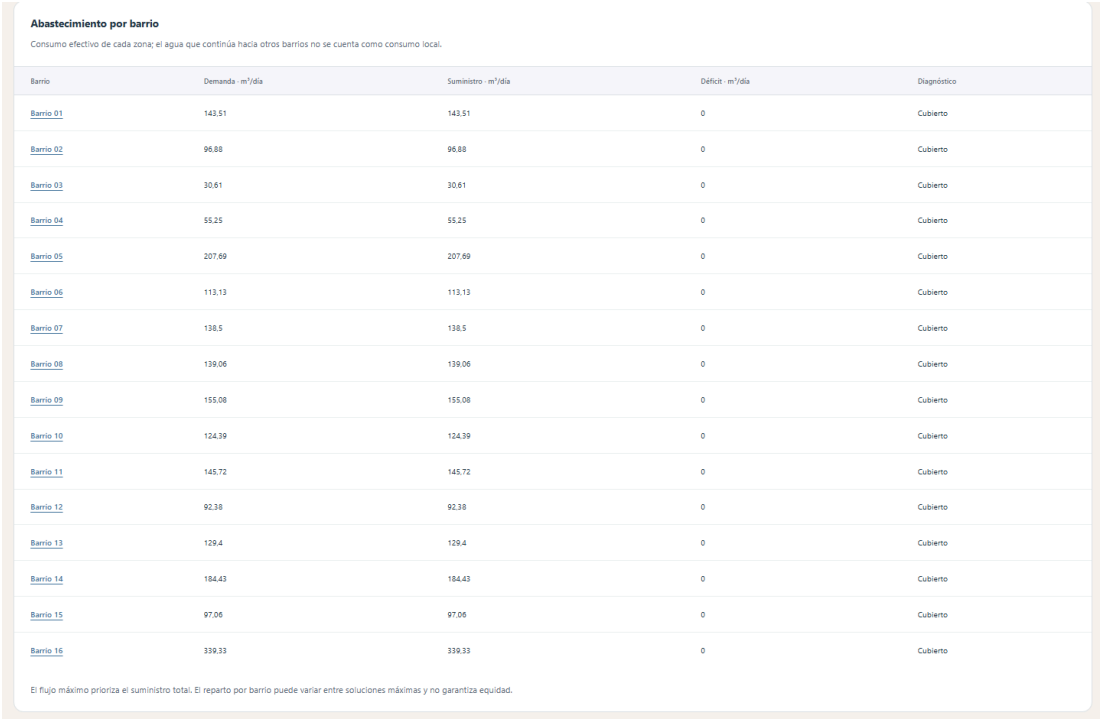


Figura 9. Caso Progresivo, +10 %: la red cubre la demanda (déficit 0) y el gráfico de flujo y capacidad por tendido.

La tabla de abastecimiento por barrio (Figura 10) detalla, para cada barrio, la demanda, el suministro, el déficit y un diagnóstico. El suministro es el consumo efectivo de la zona: el agua que continúa hacia otros barrios no se cuenta como consumo local. En esta corrida los 16 barrios figuran como “Cubierto”, y la suma de las demandas (2.192,42 m³/día) coincide con el total del diagnóstico; el Barrio 16 (zona industrial) concentra la mayor demanda, 339,33 m³/día, y el Barrio 05 le sigue con 207,69. Como el flujo máximo prioriza el total entregado, el reparto por barrio puede variar entre soluciones máximas y no garantiza equidad.



Abastecimiento por barrio

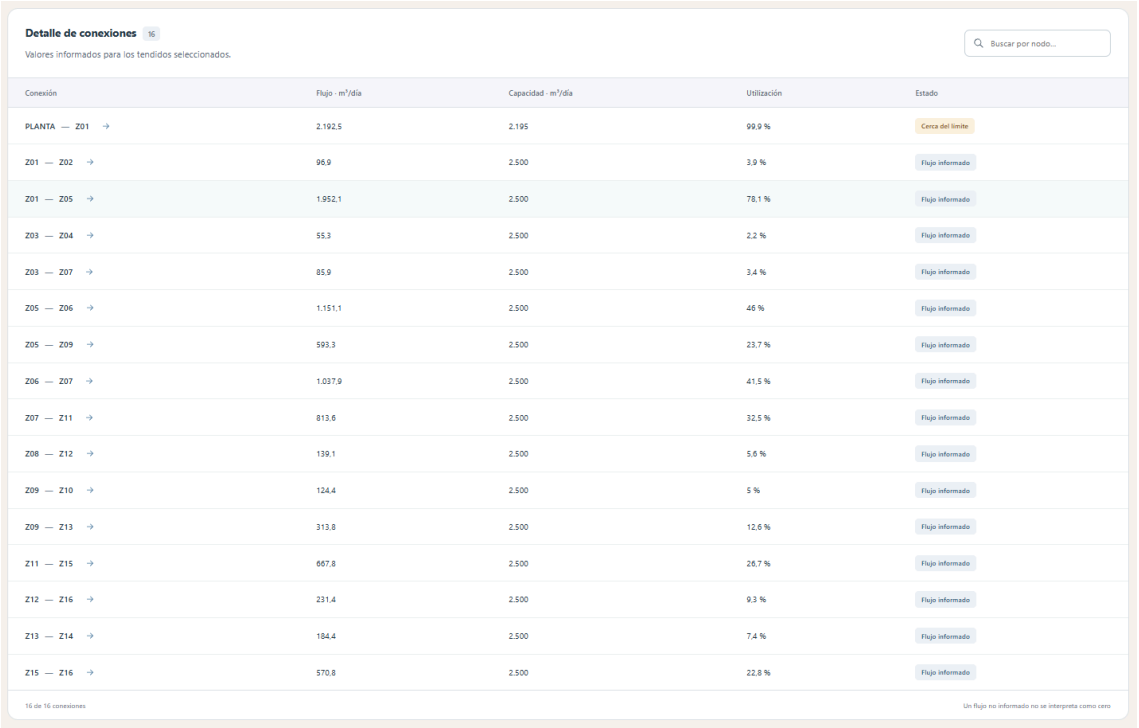
Consumo efectivo de cada zona; el agua que continúa hacia otros barrios no se cuenta como consumo local.

Barrio	Demanda · m³/día	Suministro · m³/día	Déficit · m³/día	Diagnóstico
Barrio 01	143,51	143,51	0	Cubierto
Barrio 02	96,88	96,88	0	Cubierto
Barrio 03	30,61	30,61	0	Cubierto
Barrio 04	55,25	55,25	0	Cubierto
Barrio 05	207,69	207,69	0	Cubierto
Barrio 06	113,13	113,13	0	Cubierto
Barrio 07	138,5	138,5	0	Cubierto
Barrio 08	139,06	139,06	0	Cubierto
Barrio 09	155,08	155,08	0	Cubierto
Barrio 10	124,39	124,39	0	Cubierto
Barrio 11	145,72	145,72	0	Cubierto
Barrio 12	92,38	92,38	0	Cubierto
Barrio 13	129,4	129,4	0	Cubierto
Barrio 14	184,43	184,43	0	Cubierto
Barrio 15	97,06	97,06	0	Cubierto
Barrio 16	339,33	339,33	0	Cubierto

El flujo máximo prioriza el suministro total. El reparto por barrio puede variar entre soluciones máximas y no garantiza equidad.

Figura 10. Abastecimiento por barrio (caso Progresivo, +10 %): demanda, suministro, déficit y diagnóstico de cada uno de los 16 barrios.

Por último, la tabla de flujos del caso Progresivo (Figura 11) permite ver que todos los tendidos tienen capacidad de 2.500 m³/día salvo la entrada de la planta, y que el único tramo en estado “Cerca del límite” es PLANTA — Z01.



Detalle de conexiones 16

Valores informados para los tendidos seleccionados.

Buscar por nodo...

Conexión	Flujo · m³/día	Capacidad · m³/día	Utilización	Estado
PLANTA — Z01 →	2.192,5	2.195	99,9 %	Cerca del límite
Z01 — Z02 →	96,9	2.500	3,9 %	Flujo informado
Z01 — Z05 →	1.952,1	2.500	78,1 %	Flujo informado
Z03 — Z04 →	55,3	2.500	2,2 %	Flujo informado
Z03 — Z07 →	85,9	2.500	3,4 %	Flujo informado
Z05 — Z06 →	1.151,1	2.500	46 %	Flujo informado
Z05 — Z09 →	593,3	2.500	23,7 %	Flujo informado
Z06 — Z07 →	1.037,9	2.500	41,5 %	Flujo informado
Z07 — Z11 →	813,6	2.500	32,5 %	Flujo informado
Z08 — Z12 →	139,1	2.500	5,6 %	Flujo informado
Z09 — Z10 →	124,4	2.500	5 %	Flujo informado
Z09 — Z13 →	313,8	2.500	12,6 %	Flujo informado
Z11 — Z15 →	667,8	2.500	26,7 %	Flujo informado
Z12 — Z16 →	231,4	2.500	9,3 %	Flujo informado
Z13 — Z14 →	184,4	2.500	7,4 %	Flujo informado
Z15 — Z16 →	570,8	2.500	22,8 %	Flujo informado

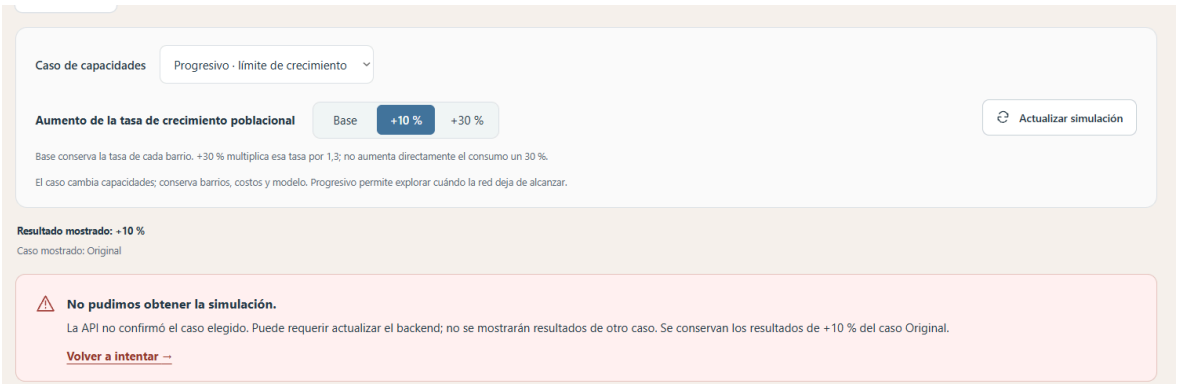
16 de 16 conexiones

Un flujo no informado no se interpreta como cero

Figura 11. Flujos informados para el caso Progresivo (+10 %): único tramo cerca del límite, PLANTA — Z01 (99,9 %).

6.5 Robustez y accesibilidad

- Estados de carga, error y reintento; ante un error de red o de contrato se conserva el último resultado válido con su identificación y no se muestra ninguno inventado.
- Si la API no confirma el caso de capacidades elegido, la interfaz no mezcla resultados: informa el error, ofrece “Volver a intentar” y conserva los resultados vigentes con su caso y escenario (Figura 12).
- Protección frente a respuestas fuera de orden: si el usuario cambia de escenario rápidamente, una respuesta antigua no pisa a la nueva.
- Distinción entre flujo cero real (se muestra 0 m³/día) y dato ausente (se muestra “Sin dato”).
- Navegación por teclado en el gráfico y en las tablas, respeto del movimiento reducido y alternativa de imagen cuando no hay WebGL en los efectos.
- Diseño adaptable: a 390 px de ancho la página no desborda horizontalmente y el gráfico se desplaza dentro de su contenedor.



Caso de capacidades: Progresivo · límite de crecimiento

Aumento de la tasa de crecimiento poblacional: Base | +10 % | +30 %

Actualizar simulación

Base conserva la tasa de cada barrio. +30 % multiplica esa tasa por 1,3; no aumenta directamente el consumo un 30 %.

El caso cambia capacidades; conserva barrios, costos y modelo. Progresivo permite explorar cuándo la red deja de alcanzar.

Resultado mostrado: +10 %

Caso mostrado: Original

No pudimos obtener la simulación.

La API no confirmó el caso elegido. Puede requerir actualizar el backend; no se mostrarán resultados de otro caso. Se conservan los resultados de +10 % del caso Original.

Volver a intentar →

Figura 12. Error al obtener la simulación del caso Progresivo (+10 %): la API no confirmó el caso, por lo que se conservan los resultados del caso Original y se ofrece reintentar.

6.6 Componentes de la interfaz

Archivo	Función
App.tsx, views/VistaTendidos.tsx, views/VistaFlujo.tsx	Estructura general y las dos vistas principales (Tendidos y Capacidad).
components/MapaRed.tsx	Gráfico SVG de la red, con selección de nodos y tendidos por mouse o teclado.
components/SelectorEscenario.tsx	Selector de caso y de escenario (Base, +10 %, +30 %) y botón para actualizar la simulación.
components/PanelDiagnostico.tsx, DiagnosticoCapacidad.tsx	Indicadores y diagnóstico de abastecimiento (demanda, abastecido, déficit).
components/TablaAristas.tsx	Tablas de tendidos y de flujos, con búsqueda por nodo.
components/DetalleSeleccion.tsx, Leyenda.tsx	Panel lateral con el detalle del elemento seleccionado y leyenda de colores.
hooks/useSimulacion.ts	Estado de la simulación: carga, error, reintento y resultado vigente.
api/cliente.ts, api/validar.ts	Llamada a la API con cancelación y validación del JSON recibido.
lib/red.ts, data/casoRed.json	Adaptación para presentar resultados y geometría de la grilla; no calculan predicciones, árbol ni flujo.
types/red.ts	Tipos de TypeScript del contrato.
views/VistaEfectos.tsx, components/effects/	Pestaña Efectos (HoloCard) y LatticeLoader durante las solicitudes.
components/TablaBarrios.tsx	Tabla de abastecimiento por barrio: demanda, suministro, déficit y diagnóstico (Cubierto o Con déficit) de cada zona.
components/ComparacionEscenarios.tsx	Comparación de los tres escenarios (Base, +10 % y +30 %) del caso elegido, con los balances que devuelve la API; no extrapola en el navegador.

6.7 Pestaña Efectos

La tercera pestaña es puramente decorativa y no interviene en los cálculos: incluye una tarjeta HoloCard (componente de React Bits, adaptado) y un indicador LatticeLoader que aparece solo durante solicitudes reales y no representa un porcentaje de avance. La procedencia y la licencia de estos componentes se documentan en `frontend/THIRD_PARTY_NOTICES.md`.

7. Resultados

7.1 Escenario +30 %, caso Original

Los valores corresponden a las capturas de las Figuras 2 a 7.

Indicador	Valor
Costo de la red de menor costo	73.066,05 u. c.
Conexiones del diseño	16 tendidos (17 nodos)
Demanda total proyectada	2.242,09 m³/día
Caudal abastecido	1.449,41 m³/día
Déficit	792,68 m³/día
Tramos críticos (utilización ≥ 90 %)	1 (Z01 — Z05)

El cuello de botella es el tendido Z01 — Z05, con capacidad 1.200 m³/día, saturado al 100 %. En el árbol de menor costo, casi toda la zona (Z05 y todo lo que cuelga de ella) depende de ese único tramo. La planta entrega 1.449,4 m³/día a Z01; de ahí salen 1.200 hacia Z05 y 101,2 hacia Z02, y el resto cubre la demanda de Z01.

La cuenta cierra: la demanda de Z01 y Z02 es de aproximadamente 249,4 m³/día, de modo que 2.242,09 − 249,41 = 1.992,68 m³/día quedan del otro lado del tramo. Como solo pasan 1.200, el déficit es 1.992,68 − 1.200 = 792,68 m³/día, exactamente el valor informado. Es decir, el déficit lo determina el corte mínimo formado por Z01 — Z05, tal como predice el teorema de flujo máximo y corte mínimo.

Otros tendidos operan con holgura (en la corrida de las capturas, Z05 — Z06 al 50,6 % y Z06 — Z07 al 13,3 %). Varios tendidos transportan 0 m³/día: el flujo máximo maximiza el total entregado, no reparte de forma equitativa, y la red no garantiza que todos los barrios reciban agua. Que un tramo tenga flujo 0 no significa falta de dato: es una de las asignaciones máximas posibles.

7.2 Comparación de escenarios y casos

Los valores del caso Original se obtuvieron ejecutando el pipeline (`ejecutar_simulacion`) con el modelo reentrenado; el +30 % coincide con las capturas. Las filas del caso Progresivo se obtuvieron ejecutando el mismo pipeline; la de +10 % es la corrida de las Figuras 8 a 11.

Caso	Escenario	Multipl. de la tasa	Demanda (m³/día)	Abastecido (m³/día)	Déficit (m³/día)	Tramos críticos	Costo de la red
Original	Base	× 1,0	2.167,63	1.435,89	731,74	1 (Z01 — Z05)	73.066,05
Original	+10 %	× 1,1	2.192,45	1.440,40	752,05	1 (Z01 — Z05)	73.066,05
Original	+30 %	× 1,3	2.242,09	1.449,41	792,68	1 (Z01 — Z05)	73.066,05
Progresivo	Base	× 1,0	2.167,62	2.167,62	0	1 (PLANTA — Z01, 98,8 %)	73.066,05
Progresivo	+10 %	× 1,1	2.192,42	2.192,42	0	1 (PLANTA — Z01, 99,9 %)	73.066,05
Progresivo	+30 %	× 1,3	2.242,09	2.195,00	47,09	1 (PLANTA — Z01, saturado)	73.066,05

Las filas del caso Original (Base y +10 %) usan la demanda sin redondear; las del caso Progresivo usan los valores que muestra la aplicación, que suma las demandas de cada barrio ya redondeadas a dos decimales. Por eso difieren en centésimas (en +10 %, 2.192,45 frente a 2.192,42 m³/día), sin que el caso altere la demanda.

La comparación muestra dos cosas. Primero, el costo de la red no cambia con el escenario ni con el caso, como corresponde al diseño en dos etapas. Segundo, en el caso Original el déficit crece con el escenario (731,74 → 752,05 → 792,68 m³/día) pero con poca sensibilidad: el modelo traduce un aumento de 30 % en la tasa en un aumento de solo 3,4 % en la demanda, y el límite lo fija un tramo que no cambia.

7.3 Caso Progresivo, +10 %

Con capacidades internas de 2.500 m³/día y entrada de 2.195, la red cubre la demanda de 2.192,42 m³/día: los 16 barrios aparecen como “Cubierto” y el déficit es 0 (Figuras 9 y 10). El resultado queda al límite: la entrada desde la planta opera al 99,9 % y cualquier incremento de demanda por encima de 2.195 m³/día no podría entregarse (sección 5.3). Es el tipo de lectura para la que se pensó el caso: identificar cuánto crecimiento soporta la red antes de que deje de alcanzar.

7.4 Detalle sobre el determinismo

El total abastecido, el déficit y el corte mínimo son invariantes, pero la asignación de flujo por tendido no lo es. Al ejecutar el escenario +30 % con distintas semillas de hash de Python, el total fue siempre 1.449,41 m³/día, mientras que el caudal de Z05 — Z06 varió (379,79 en la corrida de las capturas y 435,96 en otras) y también cambió qué tendidos quedaron en cero. La causa es que el grafo del árbol se arma a partir de conjuntos (set) de nombres de nodos, cuyo orden de iteración cambia entre ejecuciones y altera el orden en que el BFS recorre los vecinos. Es un comportamiento esperable en flujo máximo (hay muchas soluciones óptimas), pero conviene saberlo para la defensa; ordenar los nodos con `sorted()` antes de construir la red lo volvería reproducible. El mismo razonamiento aplica al reparto por barrio.

7.5 Conclusión para la cooperativa y mejoras propuestas

La red de menor costo con las capacidades del caso Original no alcanza para la demanda proyectada en ninguno de los tres escenarios, y el déficit crece con el escenario. La causa es estructural: Kruskal ignora las capacidades, por lo que eligió un tendido barato pero de 1.200 m³/día como única entrada a casi toda la zona. Con el caso Progresivo en +10 % la red alcanza, pero con margen casi nulo en la entrada de la planta.

Las siguientes ideas surgen del análisis y **no fueron implementadas ni simuladas en el repositorio**; se presentan como propuestas de trabajo futuro:

- **Reforzar un solo tendido probablemente no alcance:** elevar la capacidad de Z01 — Z05 desplazaría el cuello de botella hacia tramos más angostos aguas abajo (por ejemplo, los de 750 m³/día), que habría que evaluar.
- **Agregar tendidos que crucen el corte mínimo:** en el caso Original (+30 %) el corte mínimo separa {PLANTA, Z01, Z02} del resto y solo Z01 — Z05 lo cruza dentro del árbol. De los 10 candidatos no usados, solo Z02 — Z03 y Z02 — Z06 unen un nodo de cada lado; los otros ocho unen nodos del mismo lado y, por el teorema de flujo máximo y corte mínimo, no aumentarían el caudal. Quedaría por simular cuánto déficit eliminan esos dos y a qué costo adicional.
- **Diseño con restricción de capacidad:** agregar aristas hasta cubrir la demanda al menor costo adicional, en lugar de diseñar por costo y validar después.

8. Pruebas y verificación

Backend: 28 pruebas (datos reproducibles y conectividad, Union-Find, Kruskal con costo conocido e invariantes del árbol, Edmonds–Karp con flujo conocido y conservación, pipeline, API y regresiones). **Frontend:** 43 pruebas (contrato, cliente HTTP, respuestas desordenadas, errores y reintento, casos de capacidades y abastecimiento por barrio), además de lint y build con TypeScript estricto. El detalle por área está en el Anexo B. Estas pruebas verifican el software; no validan la precisión del modelo sobre datos reales.

La prueba de monotonicidad es relevante para la integración: asegura que un escenario con más crecimiento nunca reduzca la demanda total, es decir, que la salida del modelo efectivamente llega al algoritmo de flujo.

Cómo ejecutarlas: `python -m unittest discover -s backend/tests -v` para el backend, y `npm test`, `npm run lint` y `npm run build` (que incluye TypeScript estricto) para el frontend.

9. Uso de inteligencia artificial

Herramientas y partes asistidas

Herramienta	Uso
Codex	Análisis de la consigna, script generador del dataset, esqueleto y desarrollo del frontend React/TypeScript, cliente de API, pruebas, integración con Render y revisión de la documentación contra el código. Integración del CSV progresivo preparado por Matías y de su generador, selector de casos, suministro y déficit por barrio desde el backend, presentación y comparación en la web, y pruebas de API, invariantes e interacción. Se conservaron el caso Original y el modelo, y los resultados se verificaron por HTTP y en el navegador.
Gemini	Apoyo en la arquitectura del backend, diseño de API y esquemas, y organización de los módulos ml/, grafos/ y api/.
Paletas de Colores	Elección de la paleta de la interfaz (#44749d, #c6d4e1, #ffffff, #ebe7e0, #bdb8ad). No es IA.
Figma	Consulta preliminar de diseños de referencia. No se creó un prototipo completo.
React Bits	Componentes visuales HoloCard y LatticeLoader, adaptados y con procedencia y licencia en frontend/THIRD_PARTY_NOTICES.md. Código externo, no generado por IA.
Proyecto web personal anterior de Ciro	Aportó patrones de organización, pruebas y documentación (TypeScript, separación de interfaz y servicios, mocks para pruebas del cliente HTTP). No se atribuye a IA la autoría de ese código previo ni se identificaron fragmentos copiados literalmente.

Herramienta	Uso
Claude (Anthropic)	Revisión final de este informe: completó los apartados pendientes con datos del repositorio, volvió a ejecutar el pipeline para confirmar las cifras del caso Progresivo y editó el documento. No modificó el código del repositorio.

Para qué no se usó la IA: la elección del problema y del caso de estudio, la decisión de separar diseño por costo y validación de capacidad, y la interpretación de los resultados fueron del grupo. Matías preparó el CSV del caso Progresivo y la regla que lo genera; Ciro definió la dirección visual, eligió la paleta y los componentes de React Bits y revisó la interfaz. Los algoritmos de Kruskal y Edmonds–Karp se hicieron con implementaciones propias y no solo llamadas a NetworkX. La IA tampoco reemplazó la lectura del código: los errores de la tabla siguiente los detectamos nosotros al leer y probar lo que proponía el asistente.

Ejemplos de prompts útiles

- “Dame una estructura del proyecto (carpetas, archivos, etc.) como para ir armando la base antes de hacer cualquier código.”
- Analizar el proyecto y la consigna, contrastarlos con la teoría y resumir qué falta y cómo conseguirlo.
- Crear la página usando la paleta proporcionada y mantener las predicciones y los algoritmos en el backend.
- Agregar pruebas de cosas que importan y detecten errores, evitando tests hechos únicamente para que pasen.
- Armar un contexto para agentes de código que se mantenga actualizado con las áreas de página, datos, ML, grafos y API.
- Incorporar HoloCard en una pestaña especial y un indicador LatticeLoader durante el cálculo; aplicar TechText al grafo solo si resulta adecuado y consultar otras mejoras visuales.

Errores de la IA y cómo los detectamos

Error	Cómo lo detectamos	Resolución
La codificación one-hot dependía del lote: con drop_first=True una consulta de una sola zona perdía su categoría.	La misma zona industrial daba 10.113,16 m³/mes sola y 10.128,63 dentro de las 16 zonas.	En inferencia se generan todas las dummies y se alinean con las columnas del modelo entrenado; se rechazan tipos de zona inválidos. Tests de igualdad entre predicciones individuales y en lote.
Métricas de respaldo constantes cuando faltaba el archivo de evaluación.	Se simuló archivo ausente, JSON roto y valores no finitos.	Se quitaron los valores fijos: si no hay métricas válidas se devuelve null y la interfaz lo informa.
Un test de Kruskal repetía el algoritmo en lugar de llamar al código del proyecto.	Al leer el test se vio otro bucle de ordenamiento y Union-Find dentro de la prueba.	Se reemplazó por un CSV temporal y una llamada a calcular_red_menor_costo.
Una red desconectada se presentaba como resultado completo.	Un caso PLANTA–A y B–C devolvía dos componentes sin aviso.	Kruskal verifica que todos los nodos queden conectados y rechaza redes vacías o desconectadas (HTTP 503). Se agregaron tests de regresión.
Prueba del loader bloqueada por el reloj simulado.	La prueba agotaba su tiempo y TypeScript marcó una opción inválida.	Se separó la activación del evento del avance del reloj y se retiró la opción inválida.
Una prueba nueva trataba el orden de los extremos como dirección de una cañería.	Falló la comparación entre candidatos y árbol: Z03–Z04 y Z04–Z03 aparecían en distinto orden, aunque son la misma conexión no dirigida.	Se normalizaron los extremos como conjunto y se mantuvieron comprobaciones independientes de costo, capacidad y árbol. No se alteró el grafo para satisfacer la prueba.
Documentación que afirmaba más de lo comprobado.	Al contrastarla con el código aparecían un cuaderno vacío descrito como análisis exploratorio, conteos viejos y una selección de modelo presentada como automática.	Se ordenaron guías e informe con el estado actual, se aclaró que la elección del modelo es explícita y se precisaron los límites del análisis de errores.

Un desacierto visual: el asistente agregó un indicador tipo “live” (“API conectada” con la hora de la última respuesta). Funcionaba, pero no aportaba a la página, y se eliminó por decisión del grupo.

Reflexión

La IA nos ayudó a pasar rápido de archivos vacíos a una aplicación integrada, a organizar el trabajo y a preparar pruebas de errores y casos borde. No nos eximió de entender el código: la presencia de una llamada a NetworkX no demuestra que el algoritmo esté implementado, un texto con métricas no prueba que se hayan reproducido, y una propuesta que funciona puede ser innecesaria. Cada uno de estos puntos lo detectamos al leer el código y probarlo.

10. Limitaciones y posibles extensiones

- **Datos sintéticos:** las métricas miden cuánto aprende el modelo de las reglas simuladas. No validan un sistema de abastecimiento real.
- **Granularidad temporal:** se usan promedios diarios de un mes convencional de 30 días; no se modelan picos horarios, estacionalidad ni incertidumbre de la predicción.
- **Hidráulica:** la red representa conectividad y capacidades, no presiones, pérdidas de carga, diámetros ni bombas.
- **Diseño en dos etapas:** el árbol de menor costo no considera la demanda. No se proponen ampliaciones ni se optimiza costo y abastecimiento a la vez.
- **Capacidades del caso Progresivo:** son supuestos didácticos para explorar el límite de la red, no dimensionamientos de una obra real.
- **Equidad:** el flujo máximo no reparte de forma equitativa; un barrio puede quedar sin suministro aunque la solución sea máxima, y el reparto por barrio puede variar entre soluciones máximas.
- **Árbol sin redundancia:** al ser un árbol, cada barrio depende de un único camino; una falla en un tendido deja incomunicada una zona completa.
- **Modelos comparados:** solo regresión lineal y árbol de decisión, sin optimización de hiperparámetros.
- **Evaluación del modelo:** una sola partición con semilla fija, sin intervalos de confianza, y con un conjunto de test que se consultó durante el desarrollo.
- **Asignación de flujo no determinista:** el total y el déficit son estables, pero qué tendido lleva cuánto flujo cambia entre ejecuciones por el orden de iteración de conjuntos de Python (sección 7.4). Se resolvería ordenando los nodos.
- **Extrapolación del modelo:** la API acepta multiplicadores de crecimiento hasta 5, mientras que el modelo se entrenó con tasas de 0,5 % a 4 %. Para los escenarios del enunciado el error es despreciable (menor a 0,6 % en el total), pero no para multiplicadores grandes.
- **Verificaciones con assert:** la comparación contra NetworkX se hace con assert, que se omite si Python corre con -O. En producción convendría una excepción explícita.
- **Datos cargados al inicio:** los CSV y el modelo se leen al importar el módulo; cambiar los datos exige reiniciar el servidor.
- **Seguridad de la demo:** CORS abierto a cualquier origen y sin autenticación; adecuado para el trabajo, no para un servicio real.
- **Cobertura del frontend:** la geometría de la grilla viene de un archivo local (casoRed.json) alineado con los CSV; si se cambia el caso, hay que regenerarlo con `frontend/scripts/sincronizar_datos.py`.

Mejoras posibles, no implementadas:

- Agregar al modelo una variable de interacción consumo × crecimiento, dado que la relación real es multiplicativa; no se evaluó en este trabajo.
- Comparar otros modelos (por ejemplo, SVR o KNN) con ajuste de hiperparámetros y varias particiones.
- Propuesta automática de refuerzos (agregar aristas de la lista de candidatos para elevar el corte mínimo) y diseño con restricción de capacidad.

- Datos reales adaptados y citados; curvas horarias de consumo; intervalos de incertidumbre en la predicción; flujo de costo mínimo o reparto equitativo del déficit; análisis de confiabilidad con redundancia.

Anexo A. Tendidos candidatos (aristas_candidatas.csv)

Las 26 aristas del caso, con distancia (haversine), costo, capacidad del caso Original y si Kruskal la incorporó a la red.

Tendido	Distancia (m)	Costo (u. c.)	Capacidad (m³/día)	En la red
PLANTA — Z01	444,8	5.499,32	2.500	Sí
PLANTA — Z02	644,6	7.995,33	2.500	No
Z01 — Z02	466,5	4.431,17	2.000	Sí
Z01 — Z05	444,8	4.147,83	1.200	Sí
Z02 — Z03	466,5	5.663,82	3.000	No
Z02 — Z06	444,8	5.094,05	2.000	No
Z03 — Z04	466,5	4.947,29	750	Sí
Z03 — Z07	444,8	4.907,12	1.200	Sí
Z04 — Z08	444,8	4.994,99	3.000	No
Z05 — Z06	466,5	4.320,69	750	Sí
Z05 — Z09	444,8	4.680,17	1.200	Sí
Z06 — Z07	466,5	4.585,10	2.000	Sí
Z06 — Z10	444,8	5.327,66	3.000	No
Z07 — Z08	466,5	5.342,63	2.000	No
Z07 — Z11	444,8	4.498,15	2.000	Sí
Z08 — Z12	444,8	4.366,69	1.200	Sí
Z09 — Z10	466,5	4.420,55	750	Sí
Z09 — Z13	444,8	4.244,95	1.200	Sí
Z10 — Z11	466,5	4.932,32	2.000	No
Z10 — Z14	444,8	4.635,85	2.000	No
Z11 — Z12	466,5	5.773,26	1.200	No
Z11 — Z15	444,8	4.706,53	3.000	Sí
Z12 — Z16	444,8	4.415,56	750	Sí
Z13 — Z14	466,5	4.345,75	2.000	Sí
Z14 — Z15	466,5	5.547,30	750	No
Z15 — Z16	466,5	4.549,18	2.000	Sí

Anexo B. Pruebas automáticas

Detalle de las áreas cubiertas por las pruebas del backend (28 en total) y del frontend (43 en total).

Prueba (backend)	Qué verifica
Generador de datos	Misma semilla produce los mismos datos y el grafo candidato es conexo.
Carga de datos	Los CSV base se leen con las columnas y cantidades esperadas.
Union-Find	Operaciones básicas y detección de ciclos.
Kruskal, caso pequeño	Costo conocido, cantidad de aristas y árbol conexo sin ciclos.

Prueba (backend)	Qué verifica
Kruskal, red real	El costo coincide con NetworkX; se llama a la función del proyecto y no a una copia.
Kruskal, red desconectada	No se presenta un bosque o un nodo aislado como red completa.
Edmonds–Karp, caso controlado	PLANTA–A–B con flujo conocido y conservación del flujo.
Edmonds–Karp, red completa	Invariantes: el flujo no supera la capacidad y se conserva en los nodos intermedios.
Features y evaluación	Preparación de variables y cálculo de MAE, RMSE, R^2 y MAPE.
Inferencia estable	La predicción de una zona sola, por lote de un tipo y en el lote completo coincide para las tres categorías.
Métricas sin valores inventados	Archivo ausente o inválido devuelve null sin impedir la simulación.
Pipeline	El escenario base se ejecuta y la demanda crece de forma monótona con el multiplicador.
API	/simular-red responde con un escenario válido, rechaza uno inválido e informa HTTP 503 si la red no puede conectarse.

Las pruebas del frontend cubren el contrato de la API, el cliente HTTP (errores, tiempos de espera, respuestas desordenadas), el manejo de datos inválidos, balances inconsistentes, interacción y efectos visuales. Con la incorporación del caso Progresivo, el abastecimiento por barrio y la comparativa de escenarios, la batería de pruebas creció a **28 en el backend** (antes 22) y a **43 en el frontend** (antes 37).

El detalle de esta ampliación abarca:

- **Backend (`test_casos_abastecimiento.py`):**
  - **Simulaciones (ASGI):** Ejecuta las seis combinaciones de casos y escenarios para verificar que los balances por barrio sean no negativos y cuadren con los totales, respetando costos y capacidades.
  - **Validación de déficit:** Asegura un déficit mayor a 700 m³/día en el caso Original; 0 m³/día en el Progresivo (Base y +10 %); y mayor a 40 m³/día en el Progresivo (+30 %).
  - **Casos controlados y errores:** Comprueba que el tránsito de agua no se confunda con consumo local, que un caso desconocido devuelva HTTP 422, y que la regla del caso Progresivo genere su CSV sin sobrescribir el original.
- **Frontend (`casos.test.tsx`):**
  - **Manejo de inconsistencias:** Asegura el rechazo de suministros excesivos, balances incorrectos o servidores que ignoran el caso solicitado.
  - **Lógica de interfaz:** Cubre el desglose del consumo por barrio y la comparación de tres balances sin alterar el flujo del escenario principal.
  - **Asincronía y resiliencia:** Valida el manejo de respuestas tardías, la cancelación de procesos al cambiar de caso y el renderizado parcial (con opción de reintento) cuando falla un escenario.